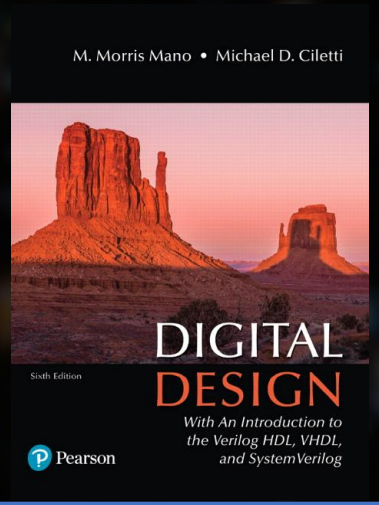
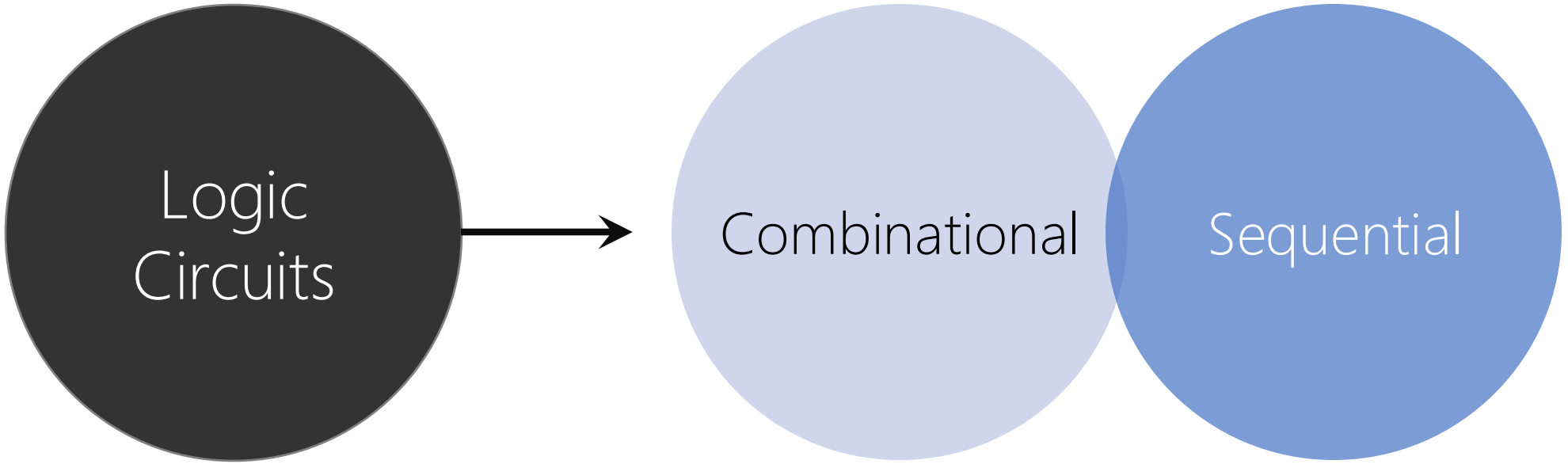


Chapter 4
Combinational Logic


Logic
Circuits
Combinational
Sequential

Chapter 4 Combinational Logic

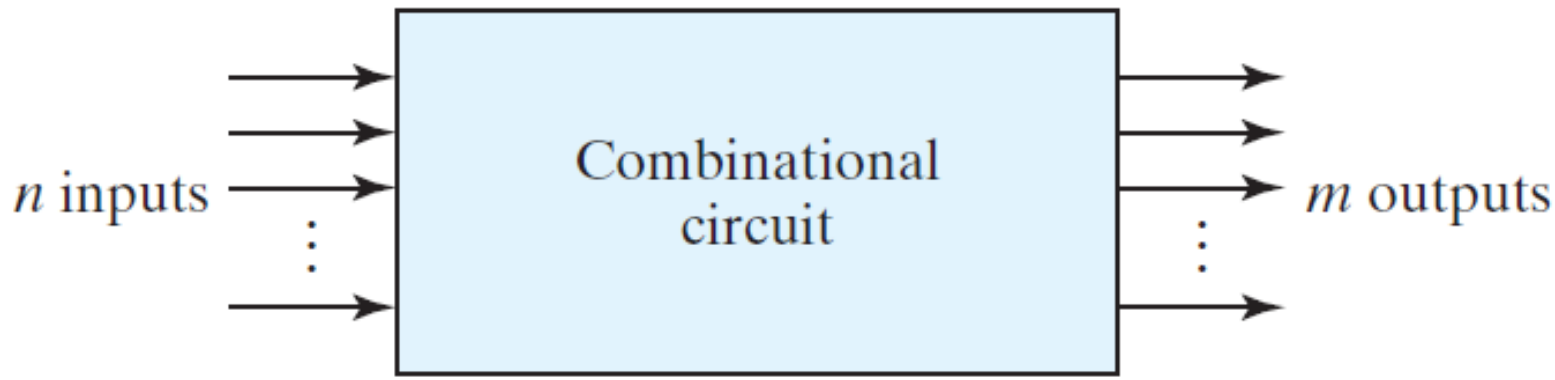
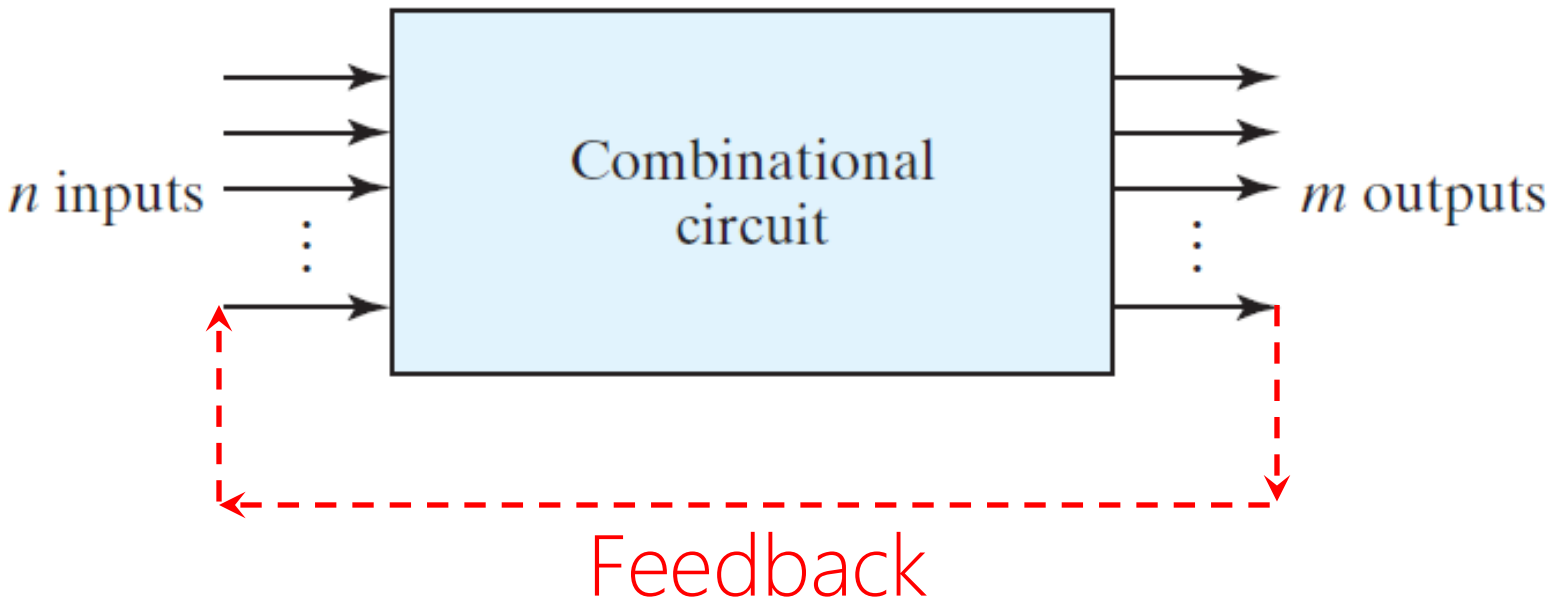


FIGURE 4.1
Block diagram of combinational circuit

Sequential Logic

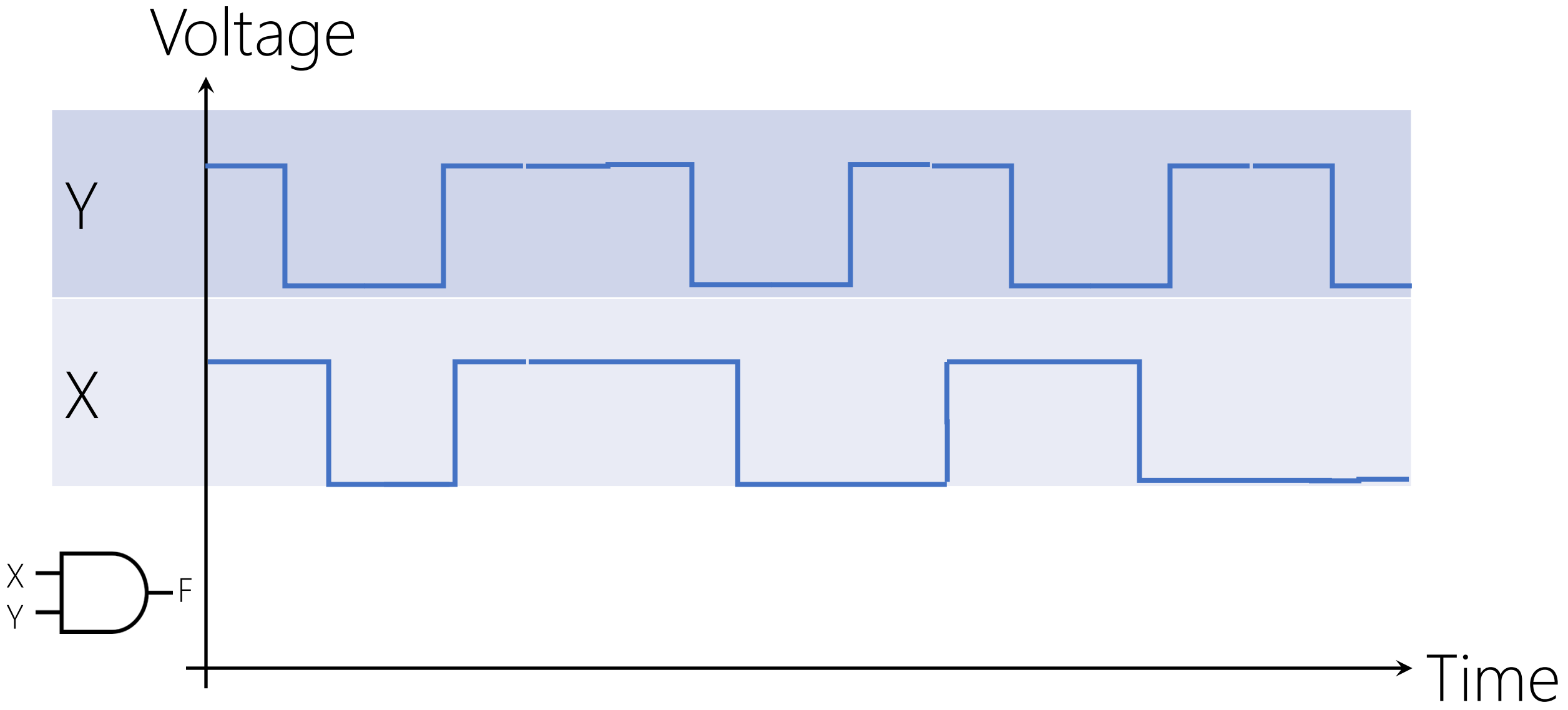


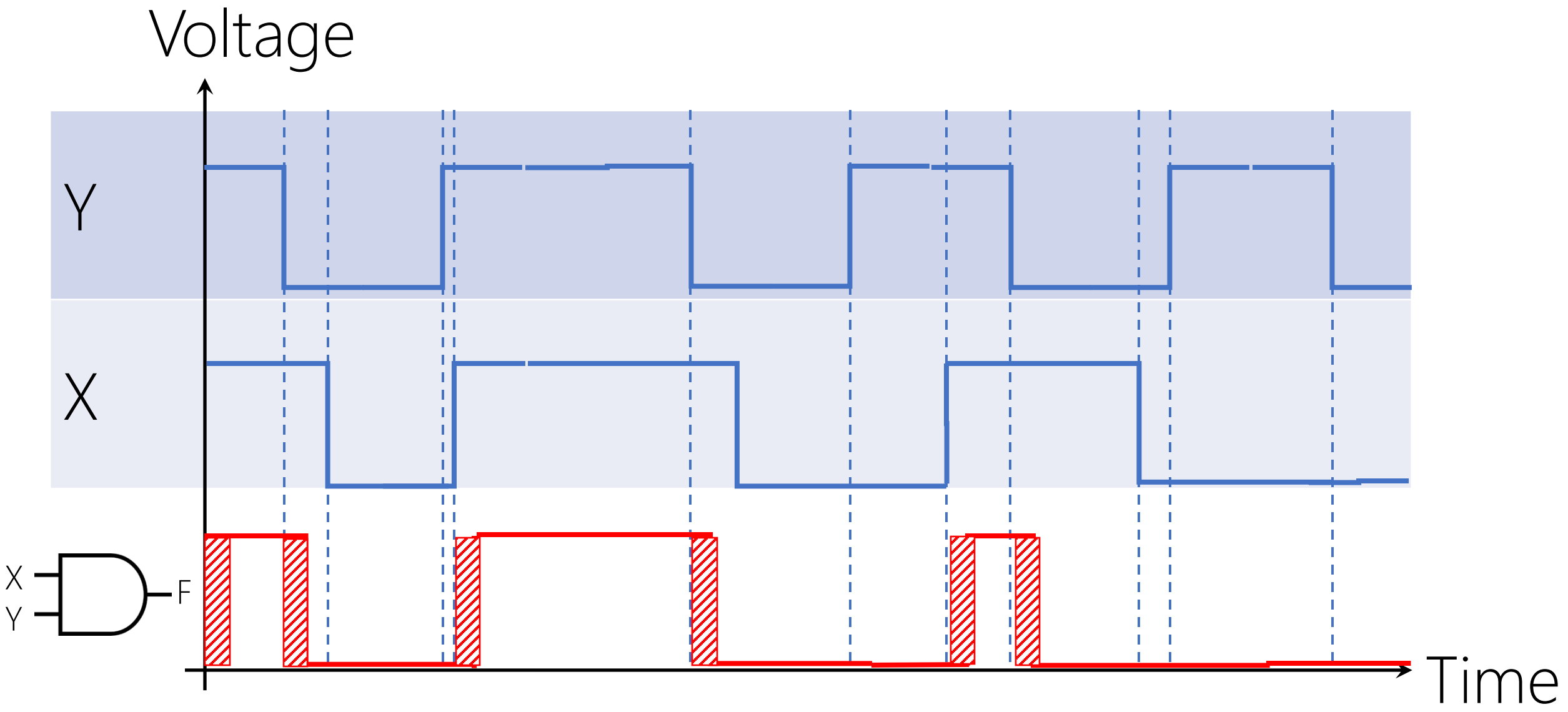
Combinational Logic

aka. Combinational Circuit

Combination of logic gates on the present inputs → the outputs *at any time*!

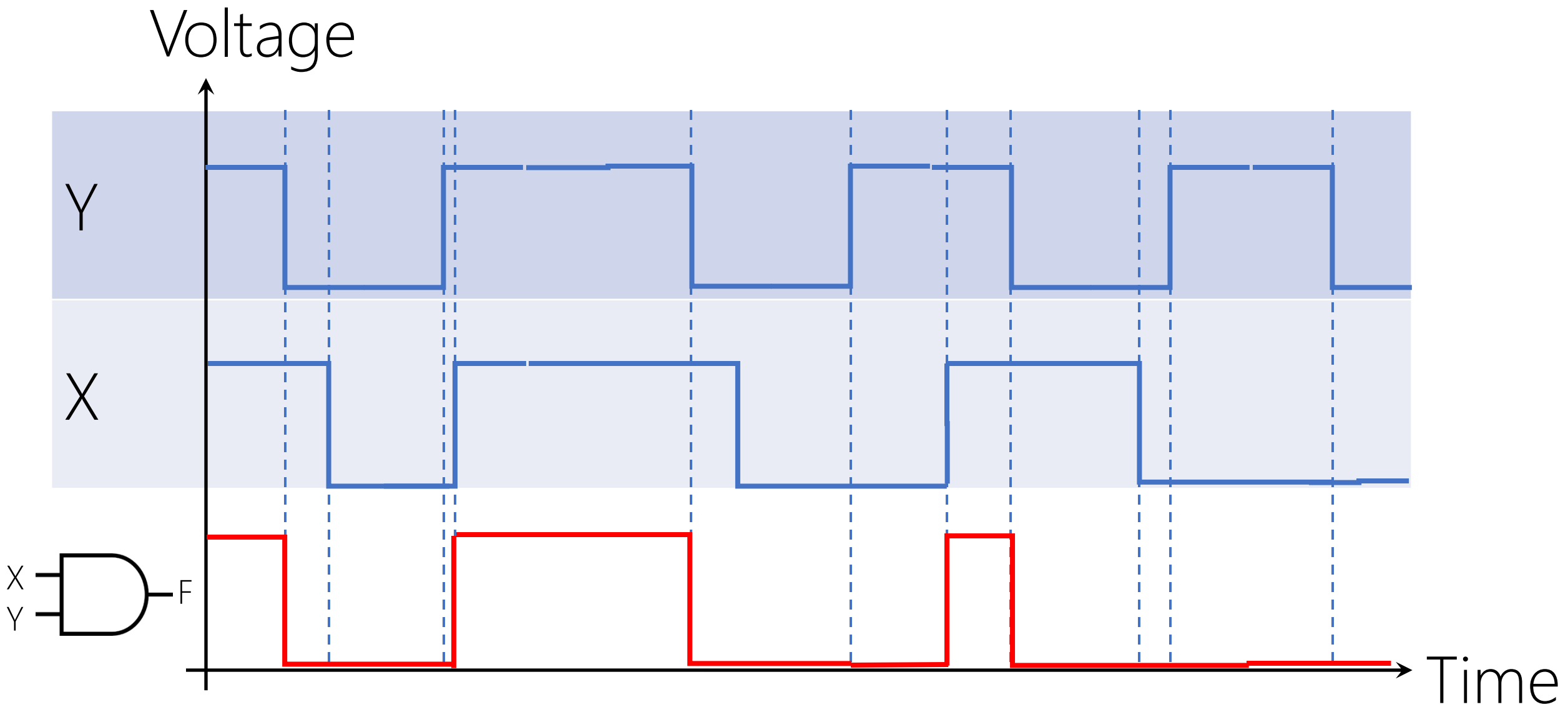
A combinational circuit performs an operation that can be specified logically by a set of Boolean functions.


Voltage
Y
X
X
Y
F
Time



Propagation Delay (Gate Delay) ≈ Δt

https://en.wikipedia.org/wiki/Propagation_delay#Electronics



Propagation Delay (Gate Delay) ≈ Δt ≈ 0

https://en.wikipedia.org/wiki/Propagation_delay#Electronics

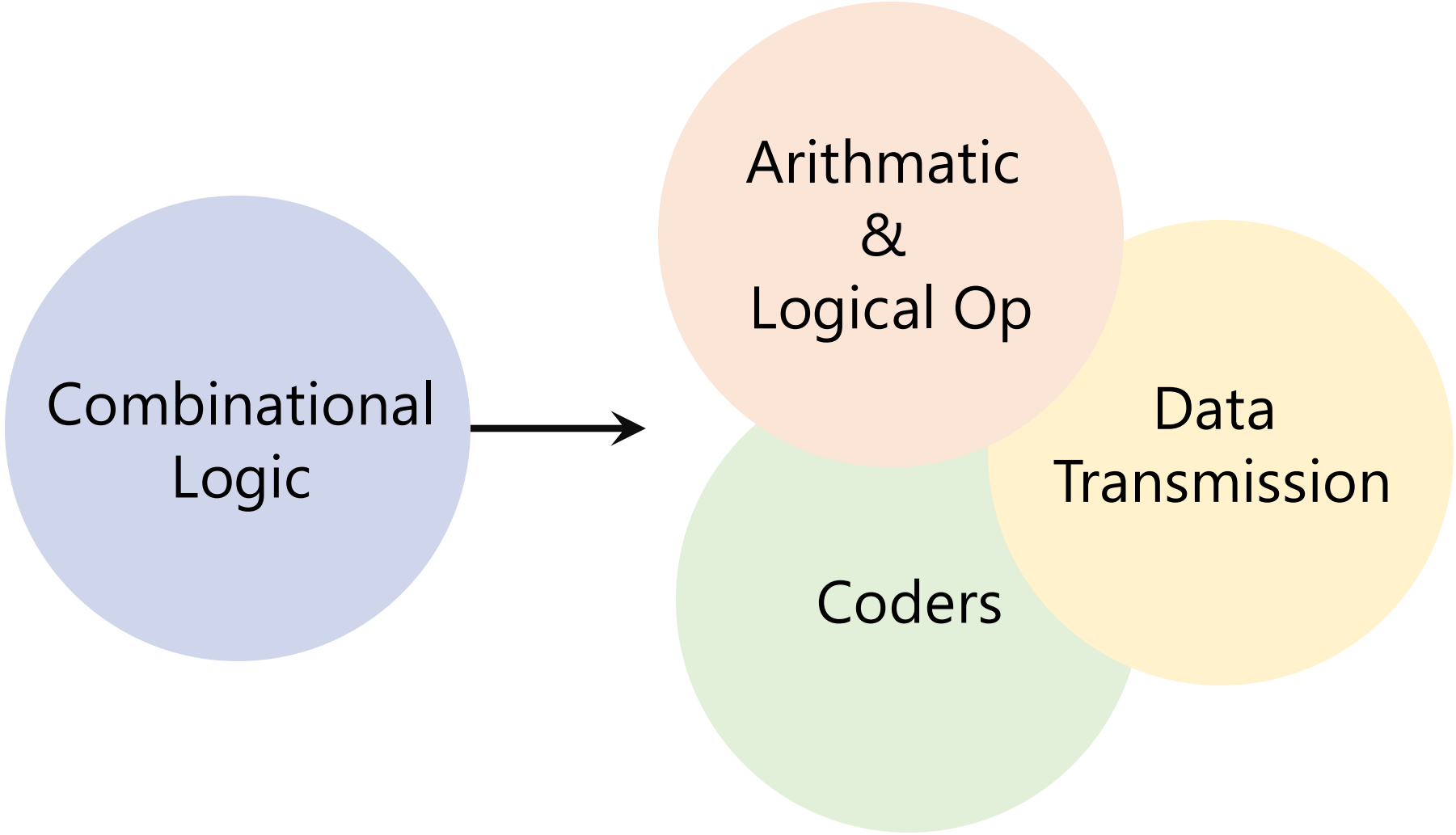
What we've done so far

Combinational Logic aka. Combinational Circuit

Design a combinational logic circuit:

1. Truth Table (Inputs, Outputs)
2. Output Boolean Functions (SoP: $\sum m$ | PoS: $\prod M$)
3. Minimization
   - Algebraically | K-Map | Quine-McCluskey
   - Double NOT >> NAND-only of SoP | NOR-only of PoS

4. Logic Diagram | Circuit


Combinational
Logic
Arithmatic
&
Logical Op
Data
Transmission
Coders

THE INTERNATIONAL
Calculator Collector

Spring 1993 Issue No. 1

The Cal Tech circa 1967 *Photo Courtesy Texas Instruments*

The Beginning

If you're past your mid-30s, you probably remember your first simple hand-held calculator costing over $50 (in early 1970's dollars). Depending how much older you are, your first could have been upwards to $400. And we're just talking the basic four functions here — addition, subtraction, multiplication, and division. Percentage and memory features were extra (if they were even available at that point in time)

Company Profile:

Bowmar

Who can forget the "Bowmar Brain" series of calculators from the early '70s?

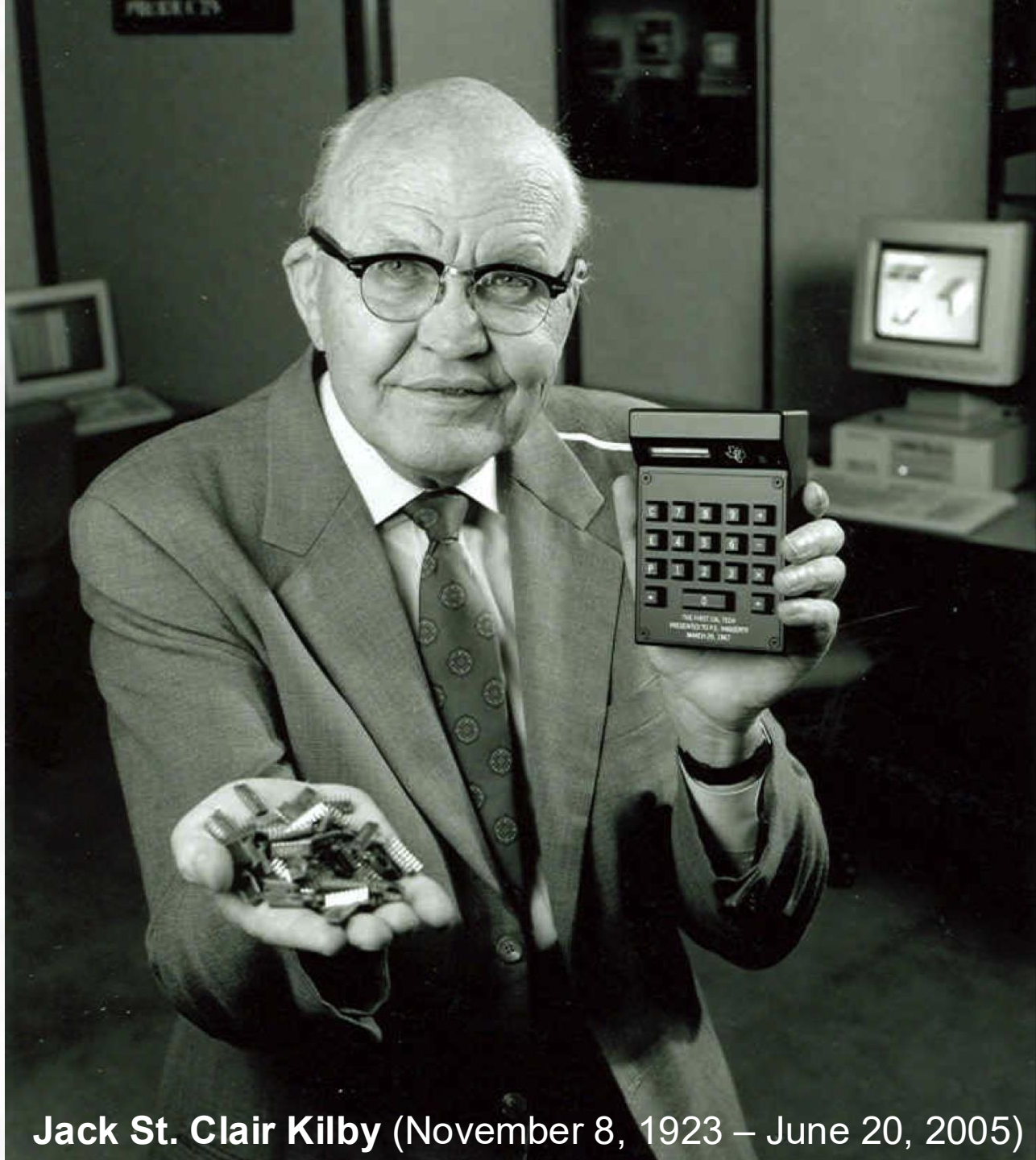
Bowmar was the first American company that made and sold their own line of portable electronic machines.

The story starts around 1970 when Bowmar, then a manufacturer of Light Emitting Diodes (LEDs), tried to sell their numeric display product to Japanese manufacturers for use in their electronic products.

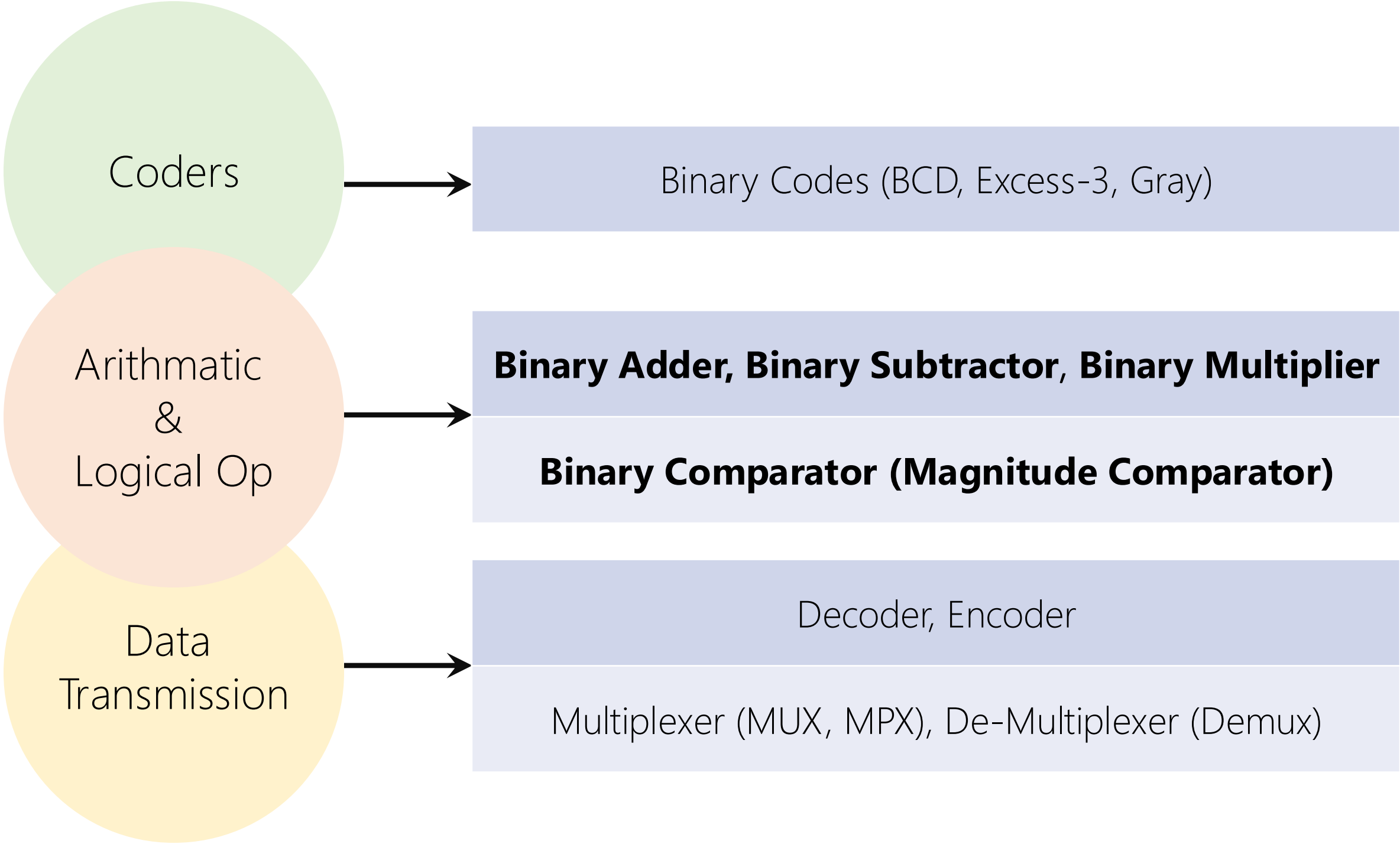
Bowmar wasn't too successful. The Japanese were using a flourescent style display that was cheaper and had a few design features the manufacturers liked better.

So, president Ed White, a consummate entrepreneur, and his staff came up with an even better idea — make the whole electronic calculator themselves.

Up to now, most of the so-called "portable" calculators



Jack St. Clair Kilby (November 8, 1923 – June 20, 2005)

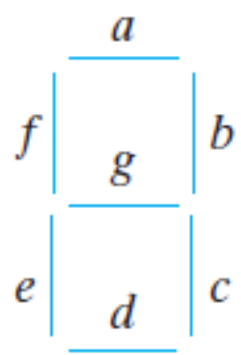

Coders
Binary Codes (BCD, Excess-3, Gray)
Arithmatic
&
Logical Op
Binary Adder, Binary Subtractor, Binary Multiplier
Binary Comparator (Magnitude Comparator)
Data
Transmission
Decoder, Encoder
Multiplexer (MUX, MPX), De-Multiplexer (Demux)

Calculator

Decimal Interface (External)
Binary Calculation (Internal)

Decimal vs. Binary Systems

4.9 An ABCD-to-seven-segment decoder is a combinational circuit that converts a decimal digit in BCD to an appropriate code for the selection of segments in an indicator used to display the decimal digit in a familiar form. The seven outputs of the decoder *(a, b, c, d, e, f, g)* select the corresponding segments in the display, as shown in Fig. P4.9(a). The numeric display chosen to represent the decimal digit is shown in Fig. P4.9(b). Using a truth table and Karnaugh maps, design the BCD-to-seven-segment decoder using a minimum number of gates. The six invalid combinations should result in a blank display. (HDL—see Problem 4.51.)

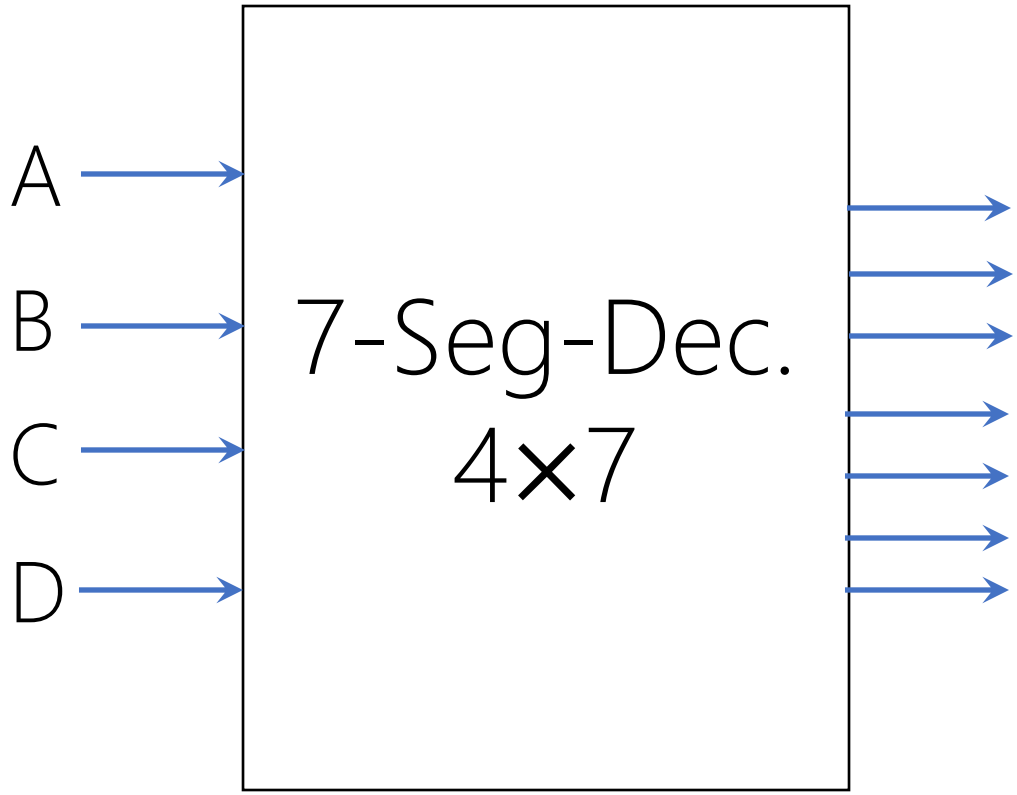


(a) Segment designation

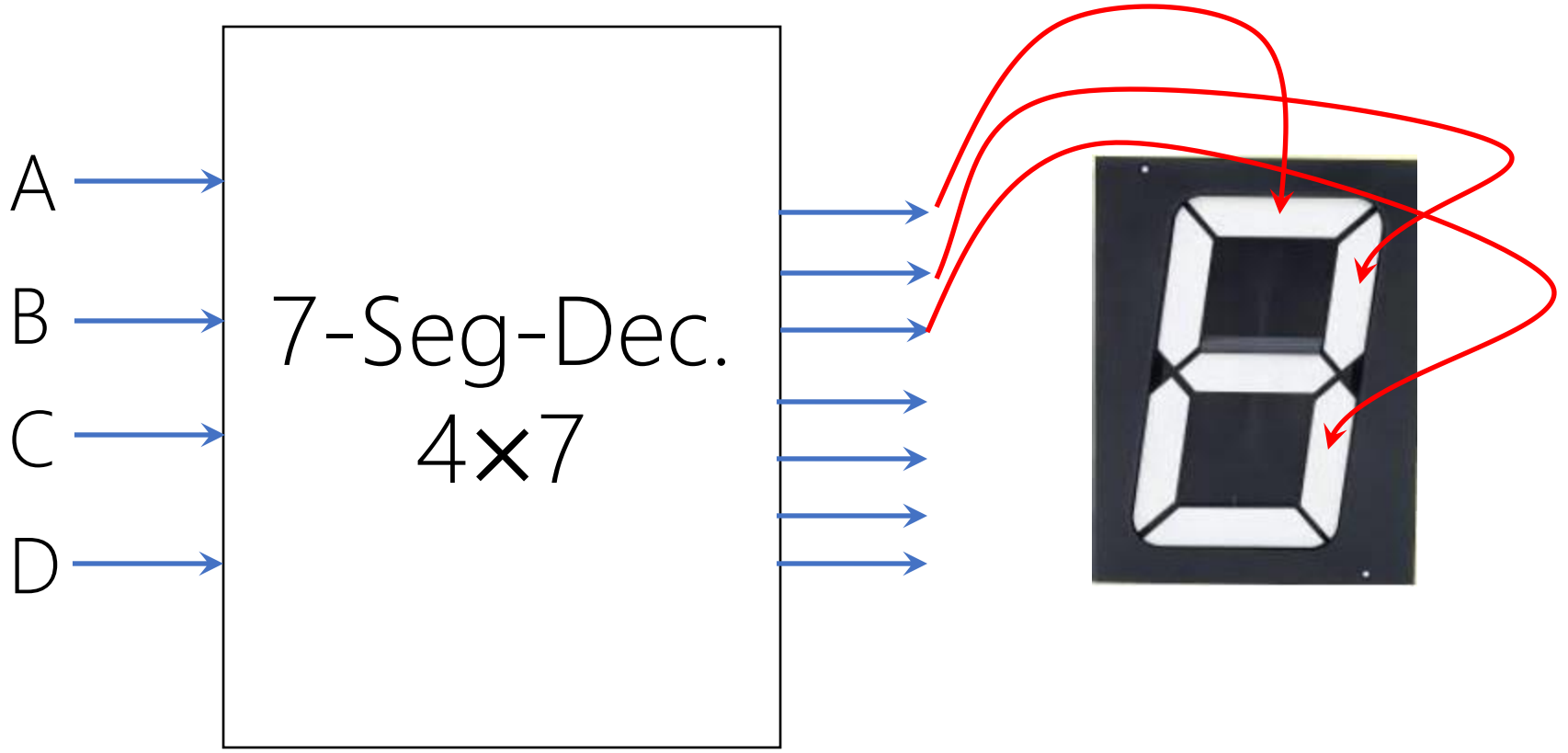
(b) Numerical designation for display

FIGURE P4.9

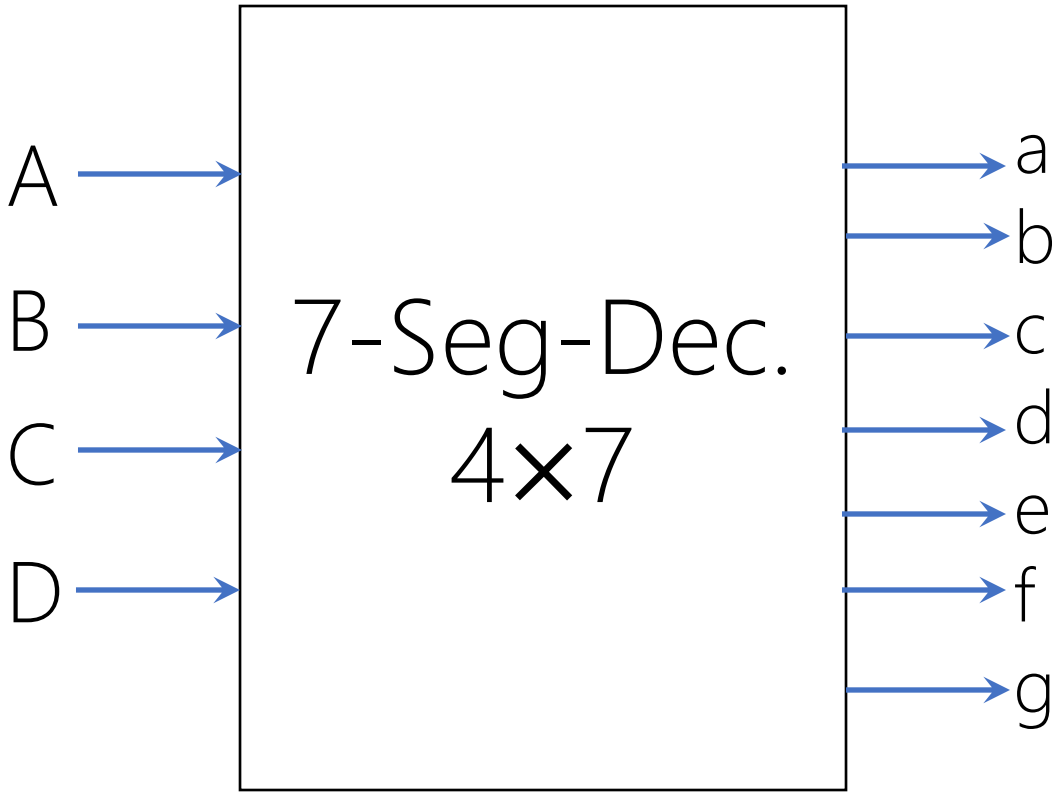
Binary Number

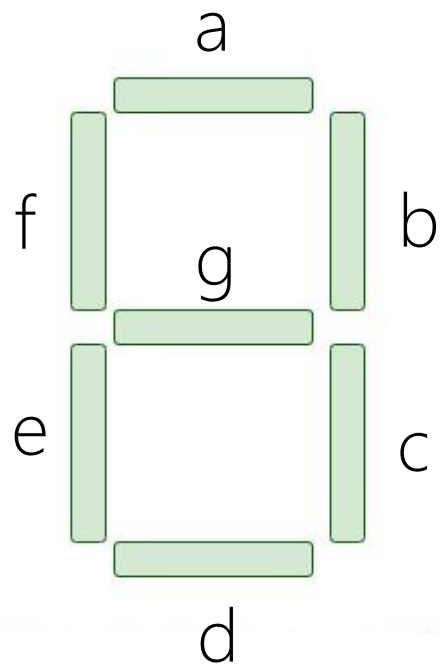


Binary Number

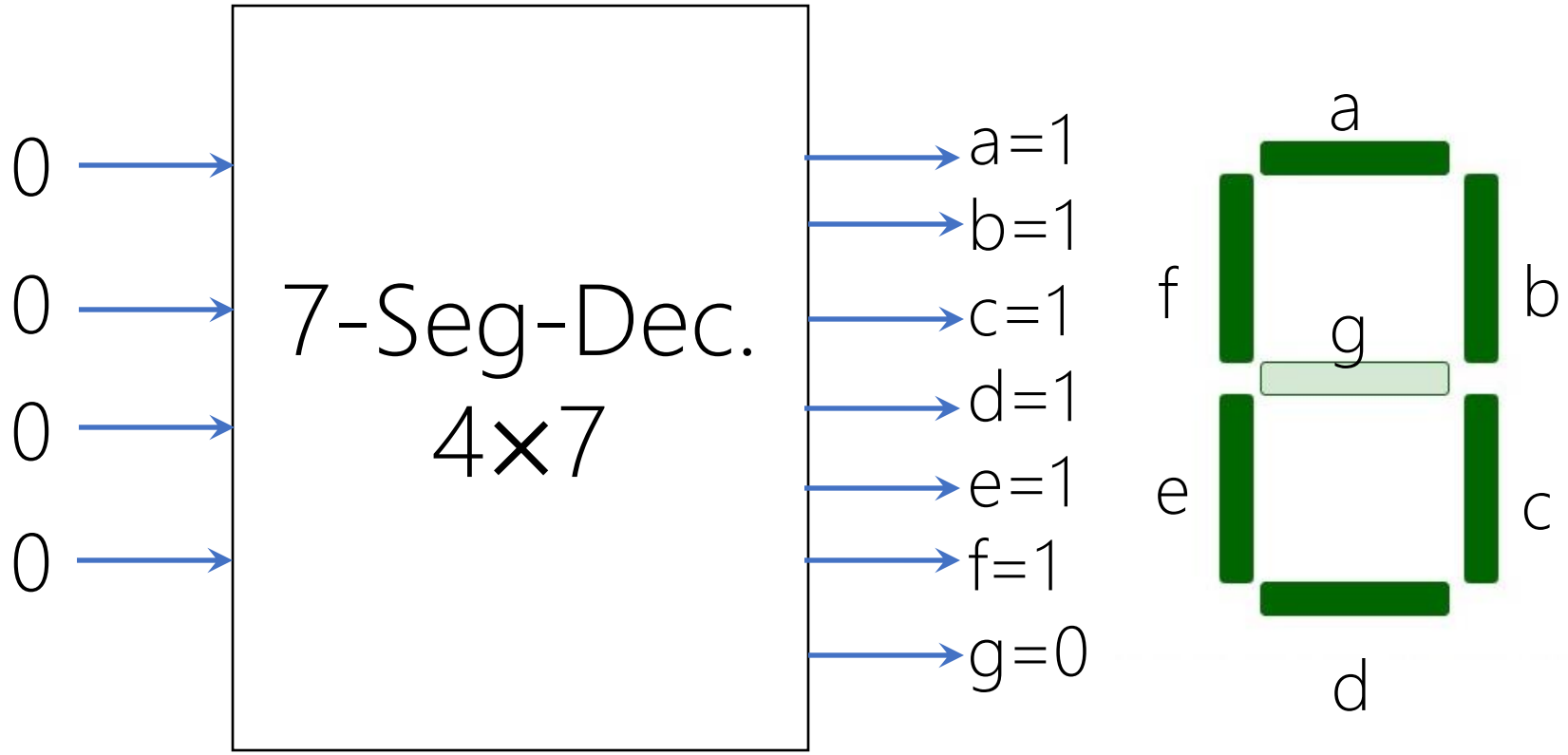


Binary Number

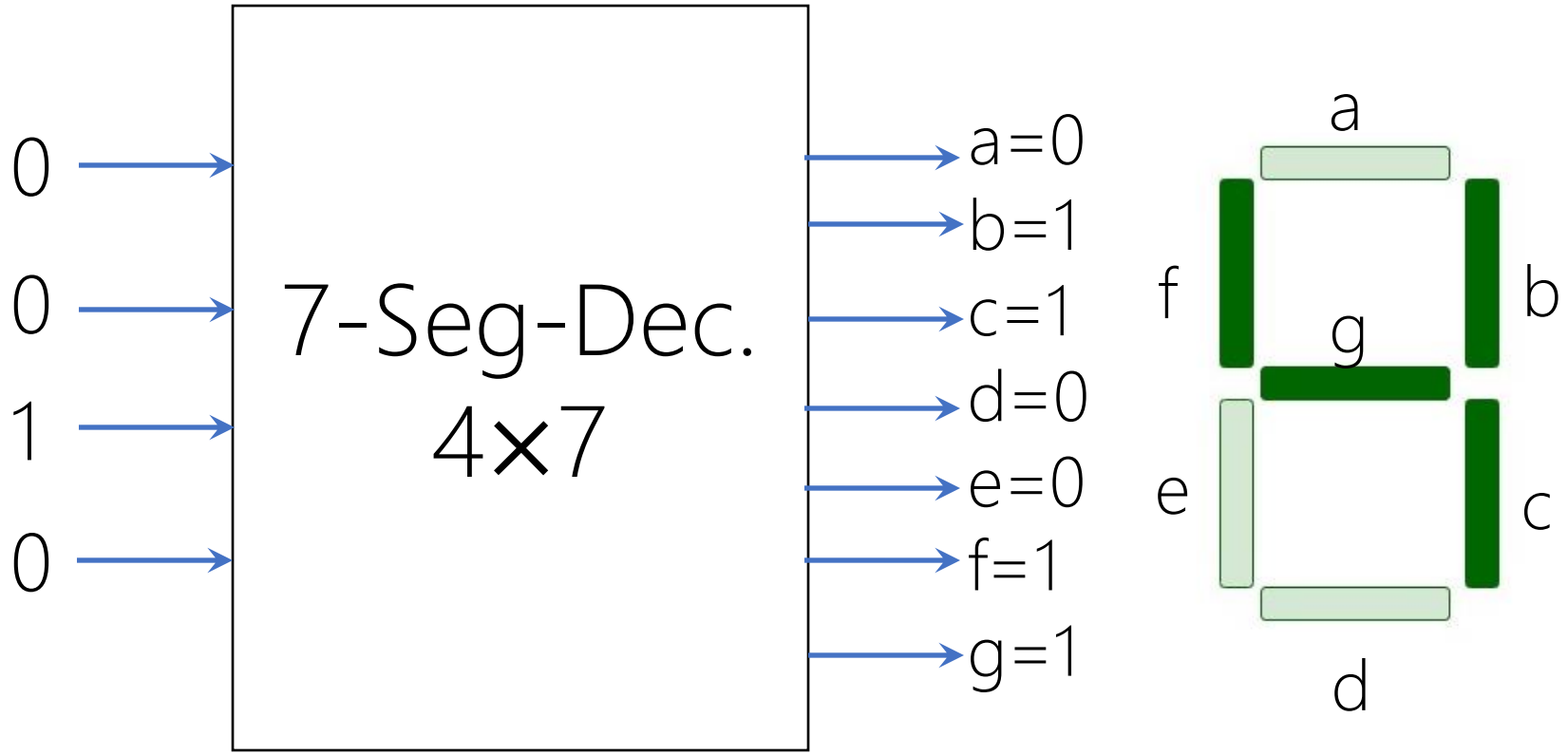




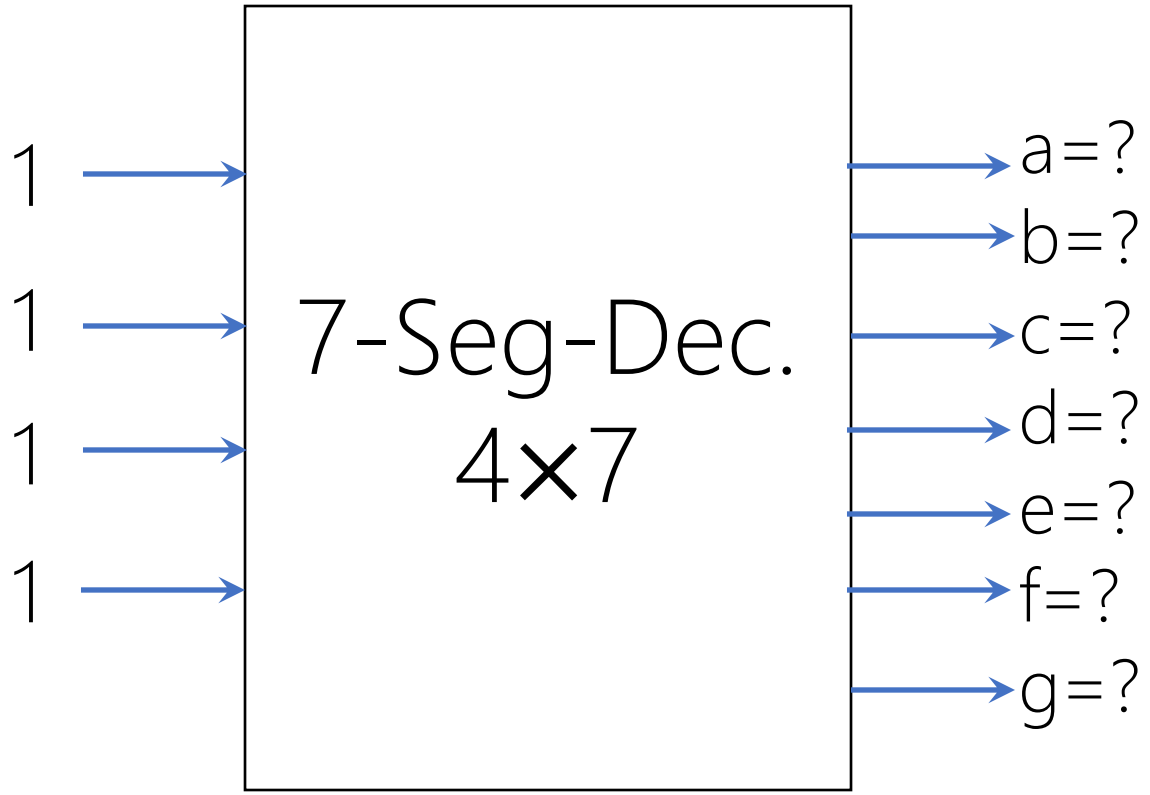
Binary Number



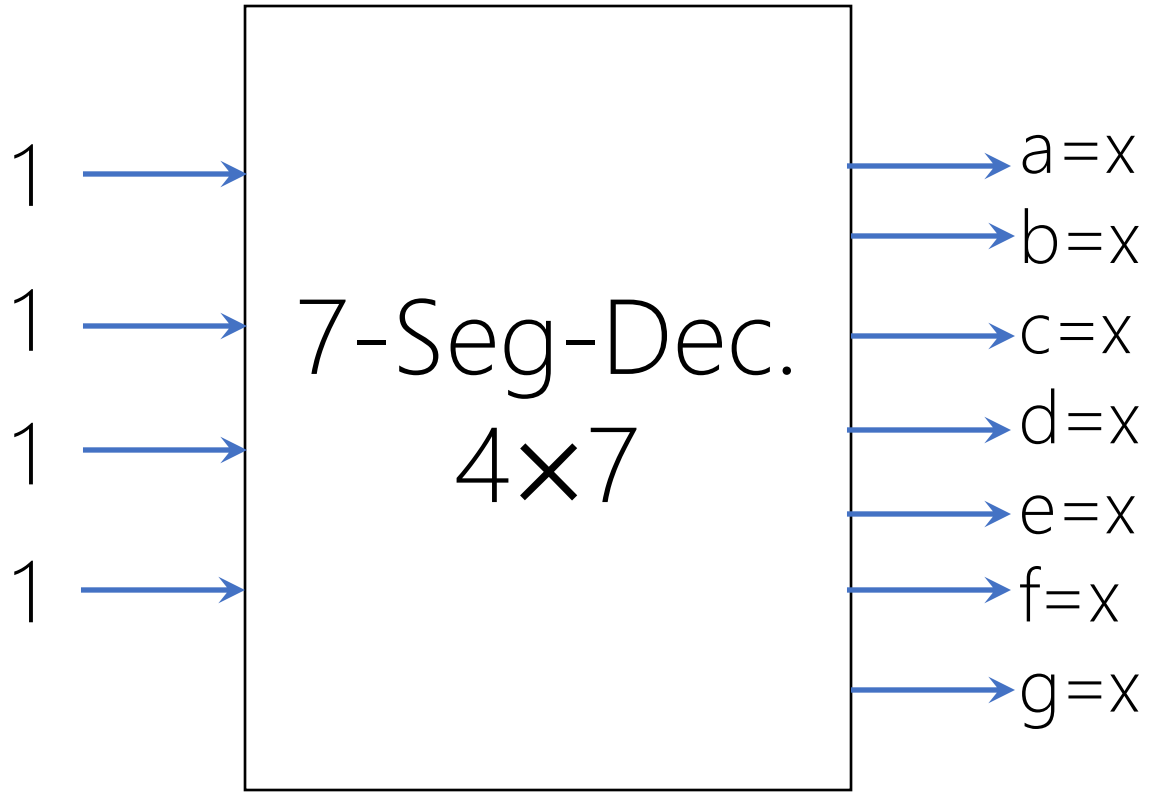
Binary Number



Binary Number



Binary Number

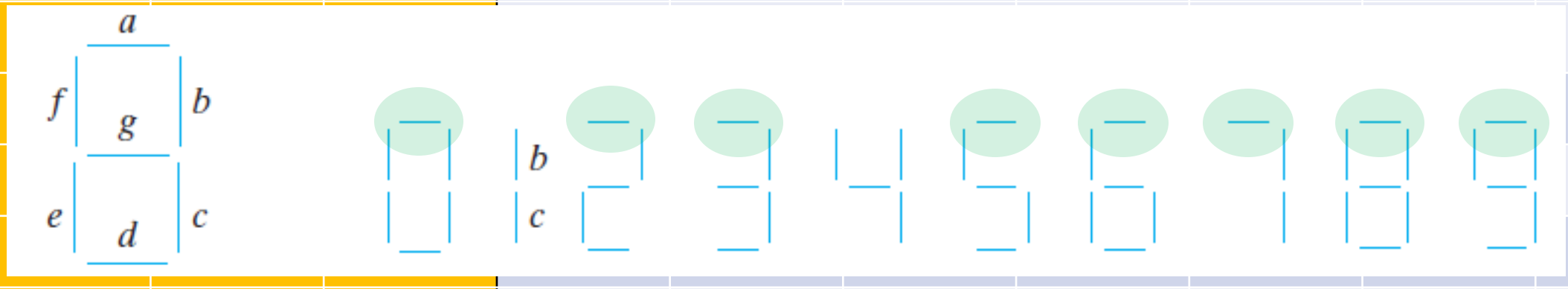


From 10 to 15, don’t care conditions!

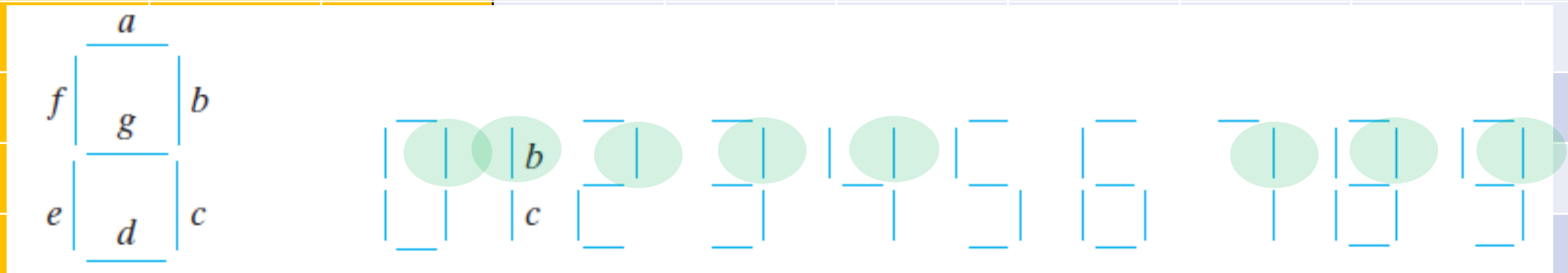
Truth Table
Display Decoder

D	C	B	A	a	b	c	d	e	f	g
0	0	0	0							
0	0	0	1							
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0	X	X	X	X	X	X	X
1	0	1	1	X	X	X	X	X	X	X
1	1	0	0	X	X	X	X	X	X	X
1	1	0	1	X	X	X	X	X	X	X
1	1	1	0	X	X	X	X	X	X	X
1	1	1	1	X	X	X	X	X	X	X

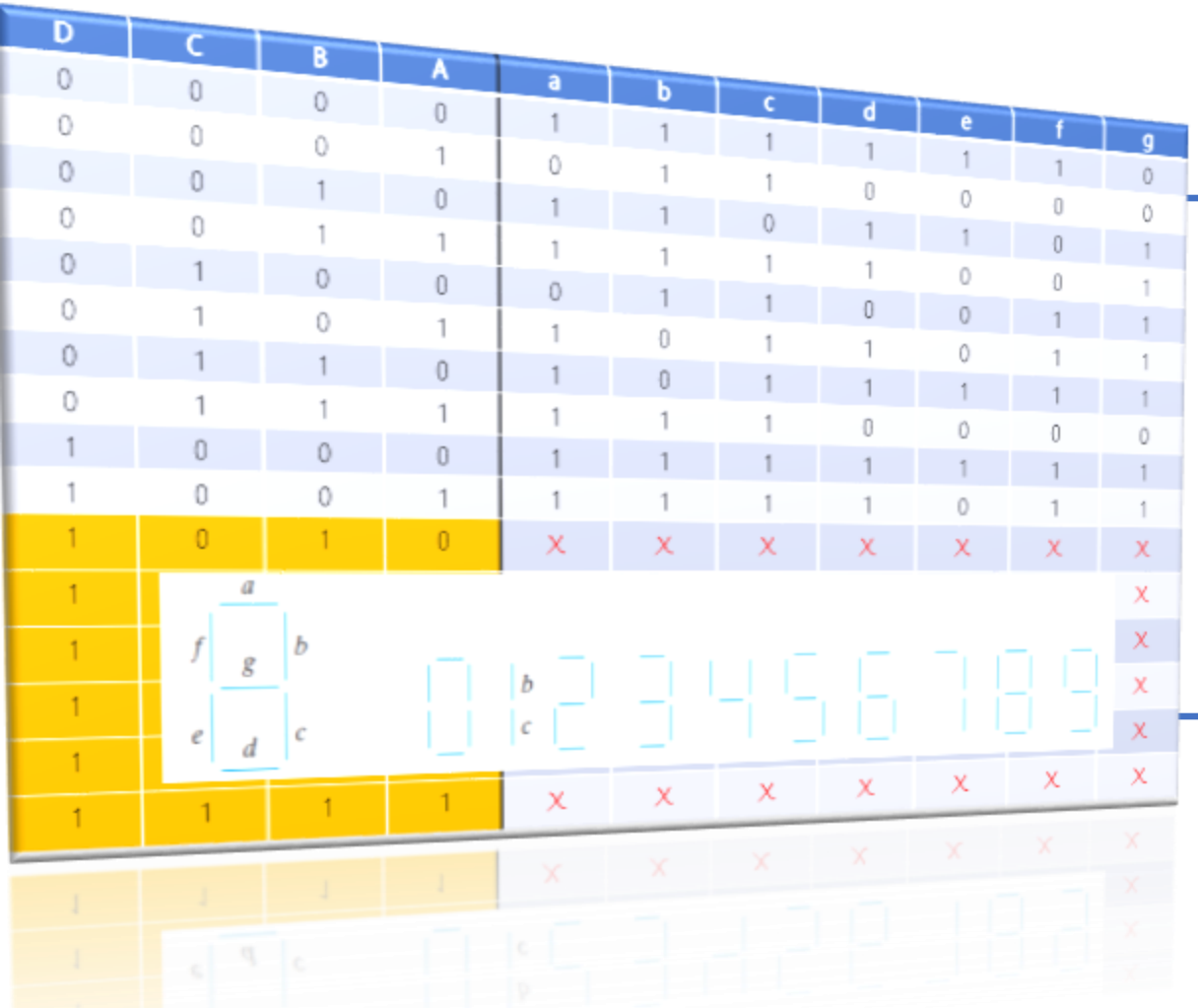
D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1						
0	0	0	1	0						
0	0	1	0	1						
0	0	1	1	1						
0	1	0	0	0						
0	1	0	1	1						
0	1	1	0	1						
0	1	1	1	1						
1	0	0	0	1						
1	0	0	1	1						
1	0	1	0	X	X	X	X	X	X	X
1										X
1										X
1										X
1										X
1	1	1	1	X	X	X	X	X	X	X



D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1					
0	0	0	1	0	1					
0	0	1	0	1	1					
0	0	1	1	1	1					
0	1	0	0	0	1					
0	1	0	1	1	0					
0	1	1	0	1	0					
0	1	1	1	1	1					
1	0	0	0	1	1					
1	0	0	1	1	1					
1	0	1	0	X	X	X	X	X	X	X
1										X
1										X
1										X
1										X
1	1	1	1	X	X	X	X	X	X	X



D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	X	X	X	X	X	X	X
1	1	0	0	X	X	X	X	X	X	X
1	1	0	1	X	X	X	X	X	X	X
1	1	1	0	X	X	X	X	X	X	X
1	1	1	1	X	X	X	X	X	X	X



D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	X	X	X	X	X	X	X
1										X
1										X
1										X
1										X
1	1	1	1	X	X	X	X	X	X	X

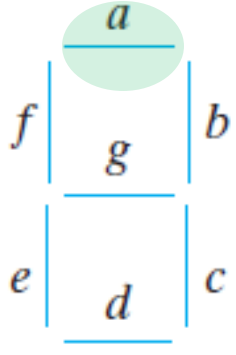
Algebraically Minimize ☹

? × ?-Variable K-Map

D	C	B	A	a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

DC \ BA	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

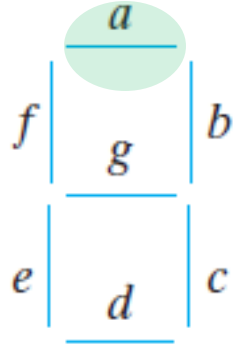
$a_{sop} = ?$



D	C	B	A	a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

DC \ BA	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

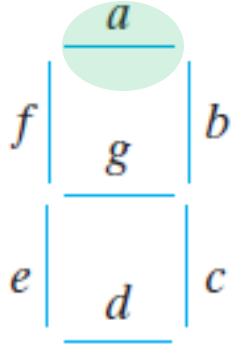
$$a_{sop} = B + DC' + D'CA + D'C'A'$$



D	C	B	A	a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

DC \ BA	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

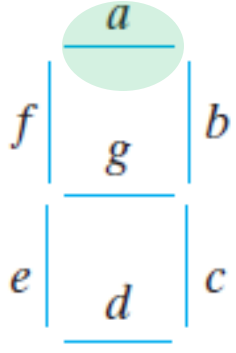
$$a_{sop} = B + D + CA + C'A'$$



D	C	B	A	a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

DC \ BA	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

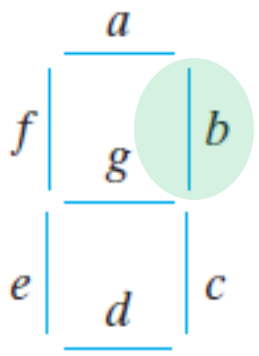
$$a_{pos} = (CB'A' + D'C'B'A)'$$
$$= (C' + B + A)(D + C + B + A')$$



D	C	B	A	b
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

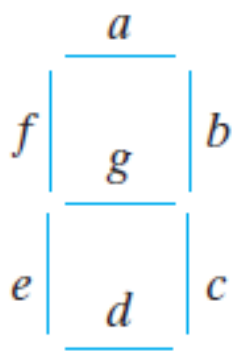
DC \ BA	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	X	X	X	X
10	1	1	X	X

$$b_{pos} = (CA)' = C' + A'$$



7 × 4-Variable K-Map

Display Decoder



At Home!

Binary Adder

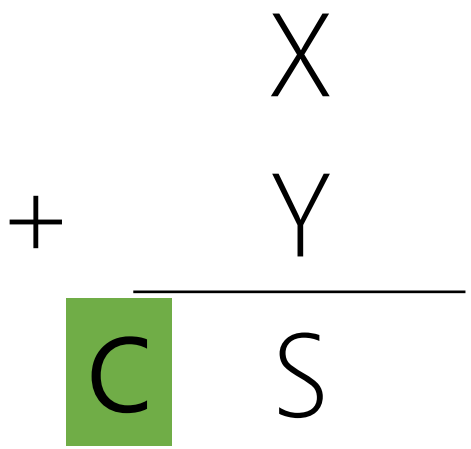
Design a logic circuit that adds two binary digits (bit).

Input binary variables: X and Y

Range of outputs?

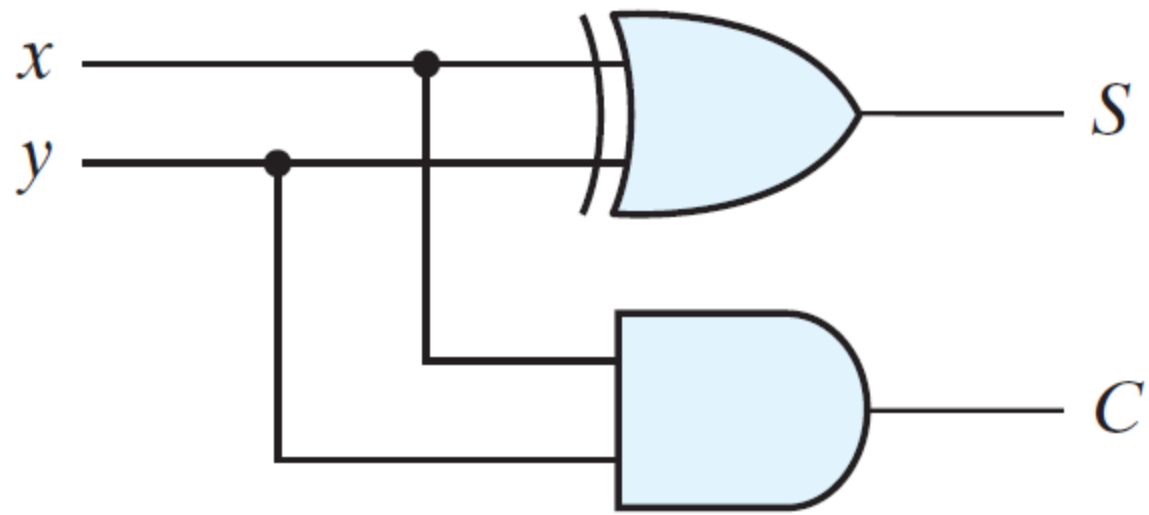
	0	0	1	1
+	0	1	0	1
	0	1	1	C=1 0

	0	0	1	1
+	0	1	0	1
	C=0 0	C=0 1	C=0 1	C=1 0

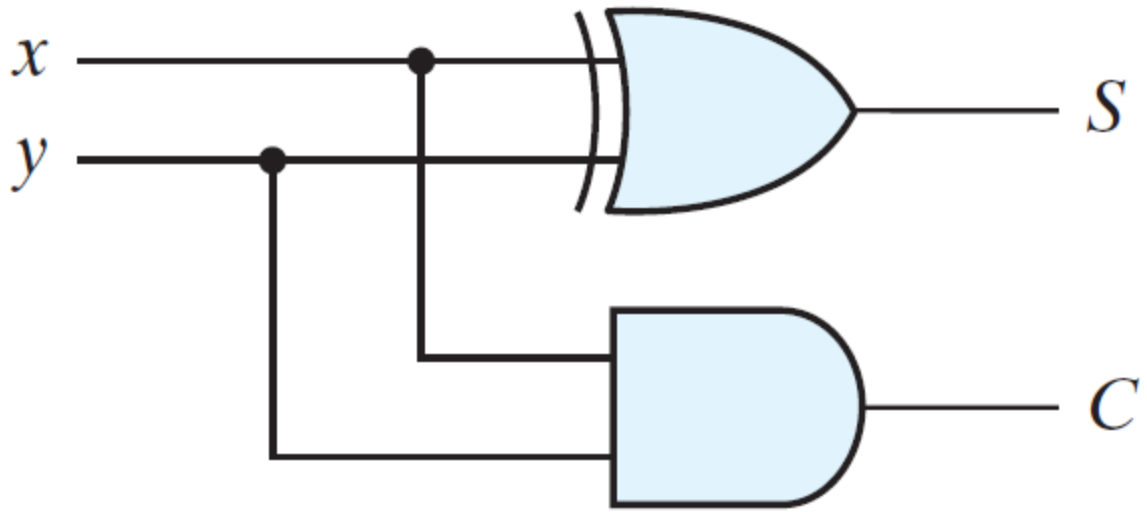

X
+ Y
C S

Output binary variables: Carry and Sum

Y	X	$F_2=C(Y,X)=YX$	$F_1=S(Y,X)=Y'X+YX'$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

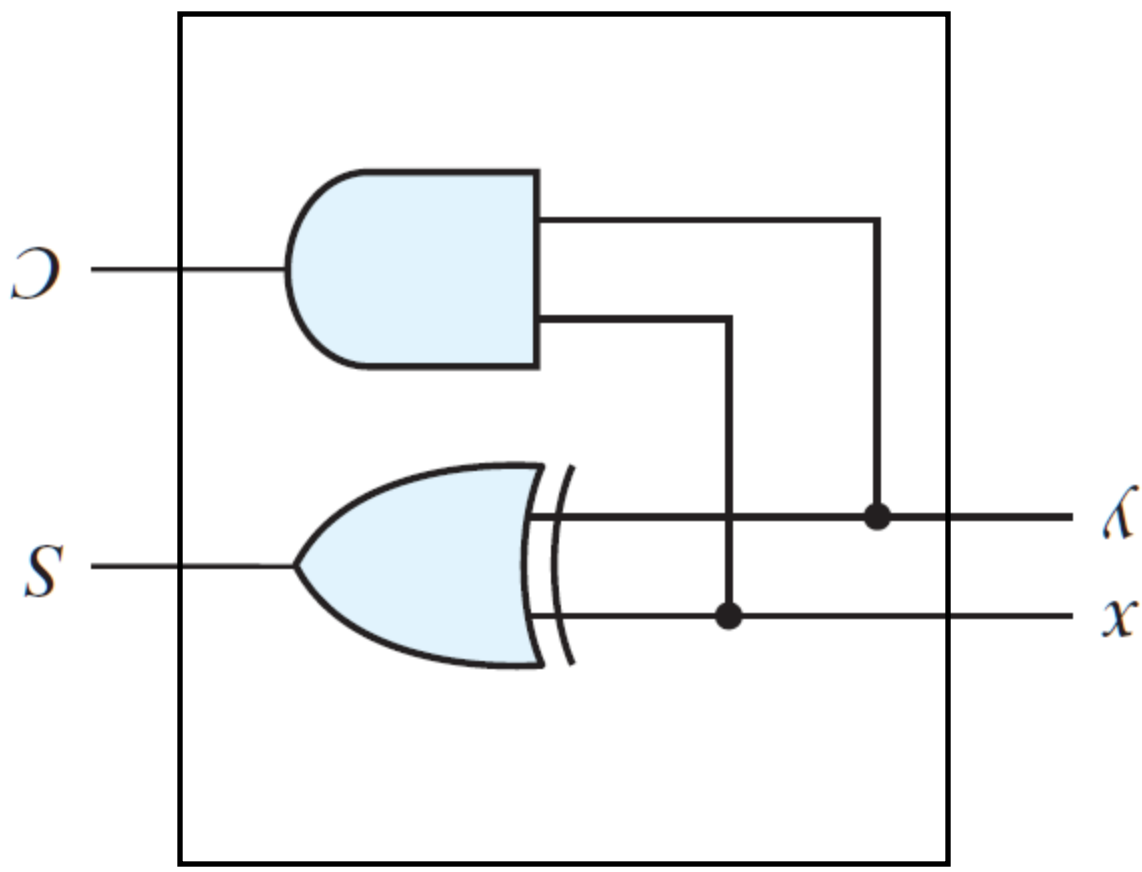


$$S = x \oplus y$$
$$C = xy$$



$$S = x \oplus y$$
$$C = xy$$

Half Adder: Just 2 bits: X+Y


C
S
y
x

y
x
Half Adder
C
S

Design a logic circuit that adds two binary numbers!

Range of inputs:
2 binary numbers in range $[00,11]_2$

Input binary variables: $X = X_2X_1$ and $Y = Y_2Y_1$

Range of outputs?

	00		00		00	...	11
+	00		01		10	...	11
C=0	00	C=0	01	C=0	10		C=1 10

$$
\begin{array}{rr}
 & X_2X_1 \\
+ & Y_2Y_1 \\
\hline
C & S_2S_1
\end{array}
$$

Range of outputs?
Carry, S_2, S_1

Y_2	Y_1	X_2	X_1	$C(Y_1,Y_2,X_2,X_1)$	$S_2(Y_1,Y_2,X_2,X_1)$	$S_1(Y_1,Y_2,X_2,X_1)$
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

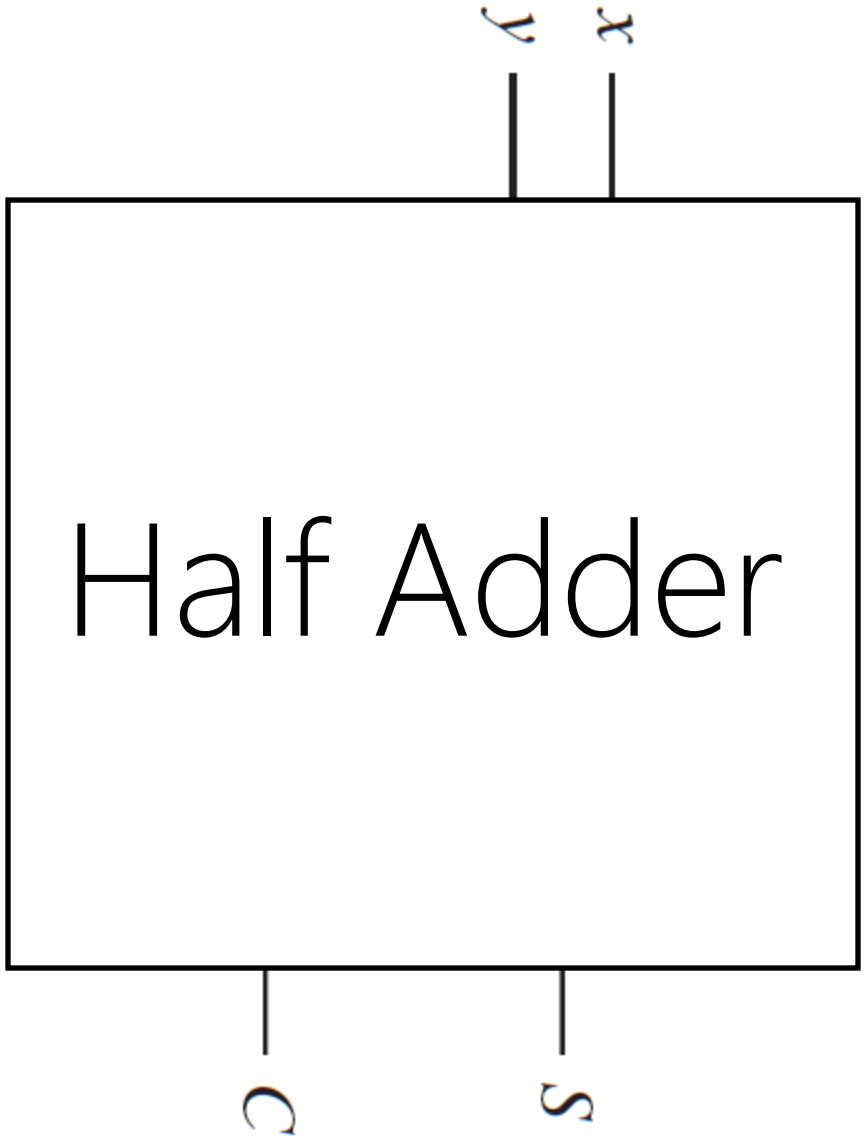
Y_2	Y_1	X_2	X_1	$C(Y_1,Y_2,X_2,X_1)$	$S_2(Y_1,Y_2,X_2,X_1)$	$S_1(Y_1,Y_2,X_2,X_1)$
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	0
0	0	1	1	0	0	1
0	1	0	0	0	0	1
0	1	0	1	0	0	0
0	1	1	0	0	0	1
0	1	1	1	0	1	0
1	0	0	0	0	0	0
1	0	0	1	0	0	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	1	0
1	1	1	0	1	1	1
1	1	1	1	1	1	0

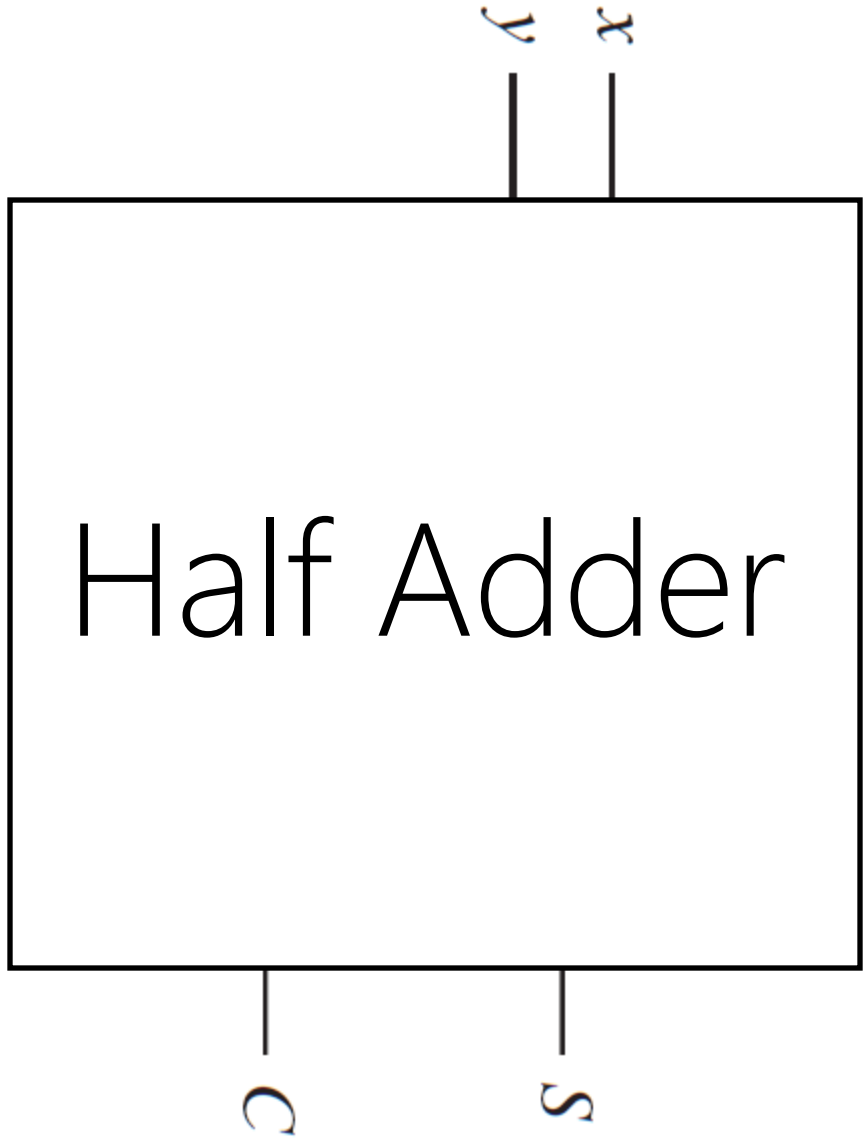
Wait a sec!

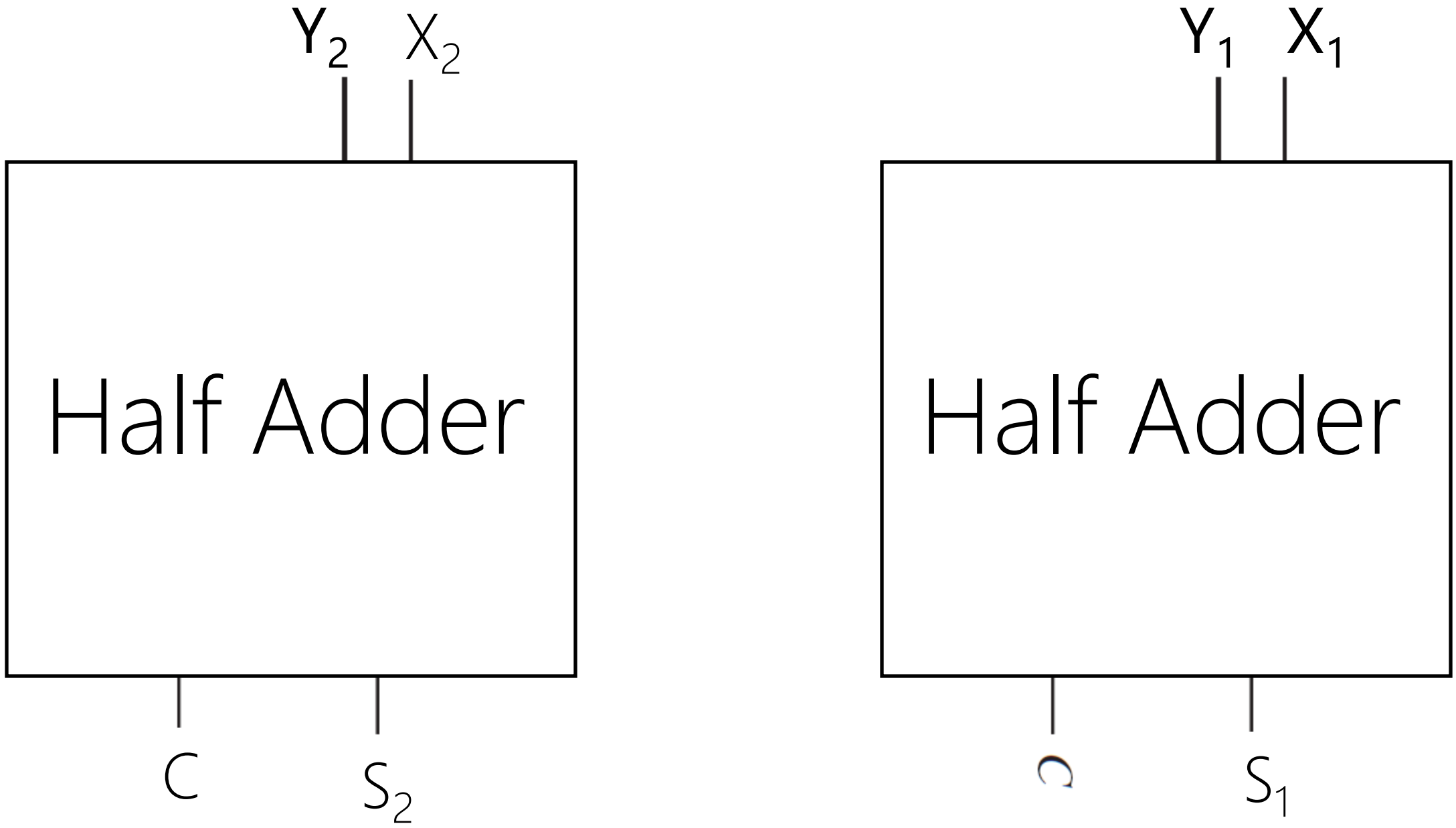
Can we re-use the half adder?

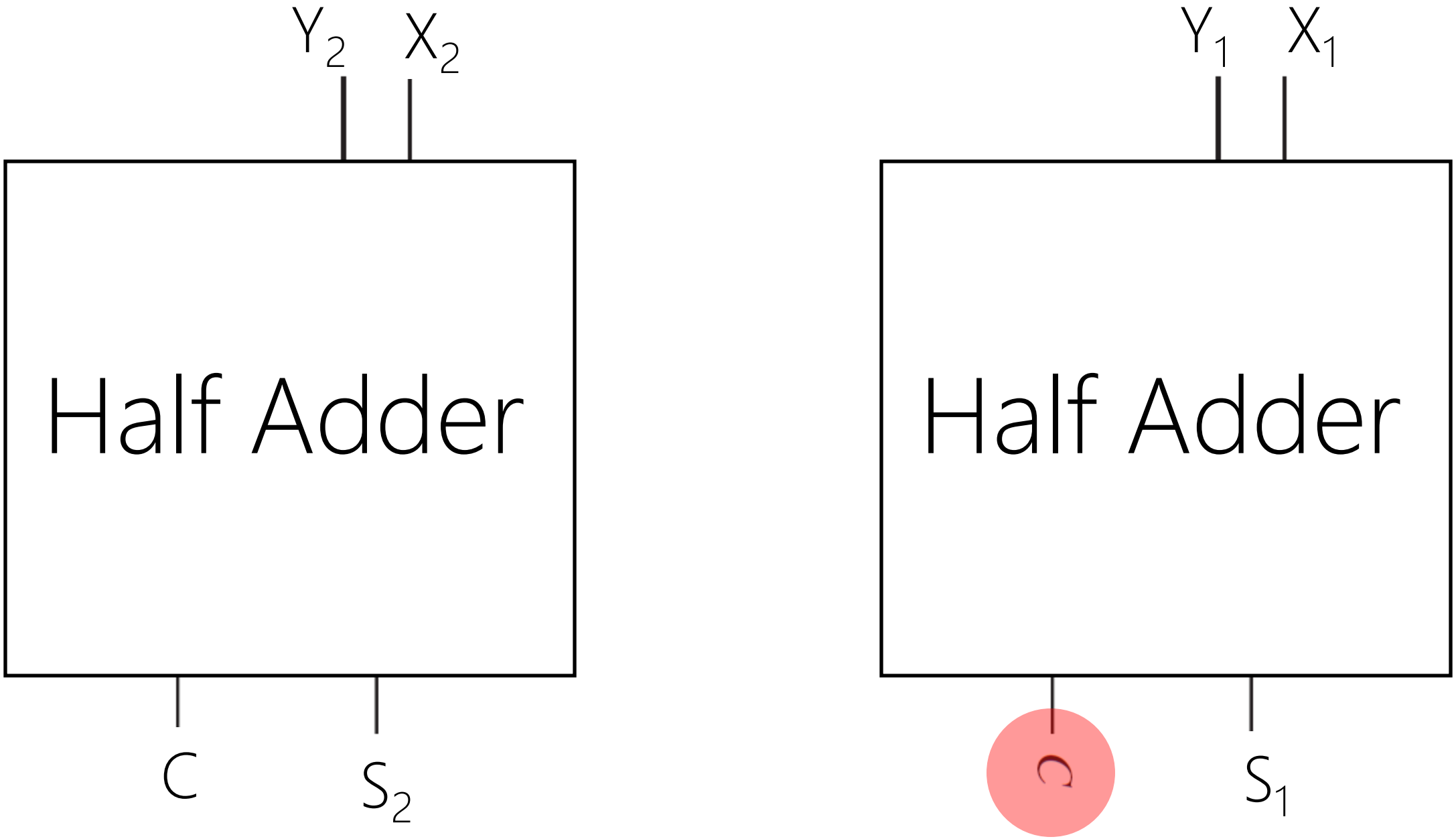
Half adder for adding 2 bits.
How about having 2 half adders for adding 2 × 2 bits?

$$
\begin{array}{rr}
 & X_2X_1 \\
+ & Y_2Y_1 \\
\hline
C & S_2S_1
\end{array}
$$


y
x
Half Adder
C
S

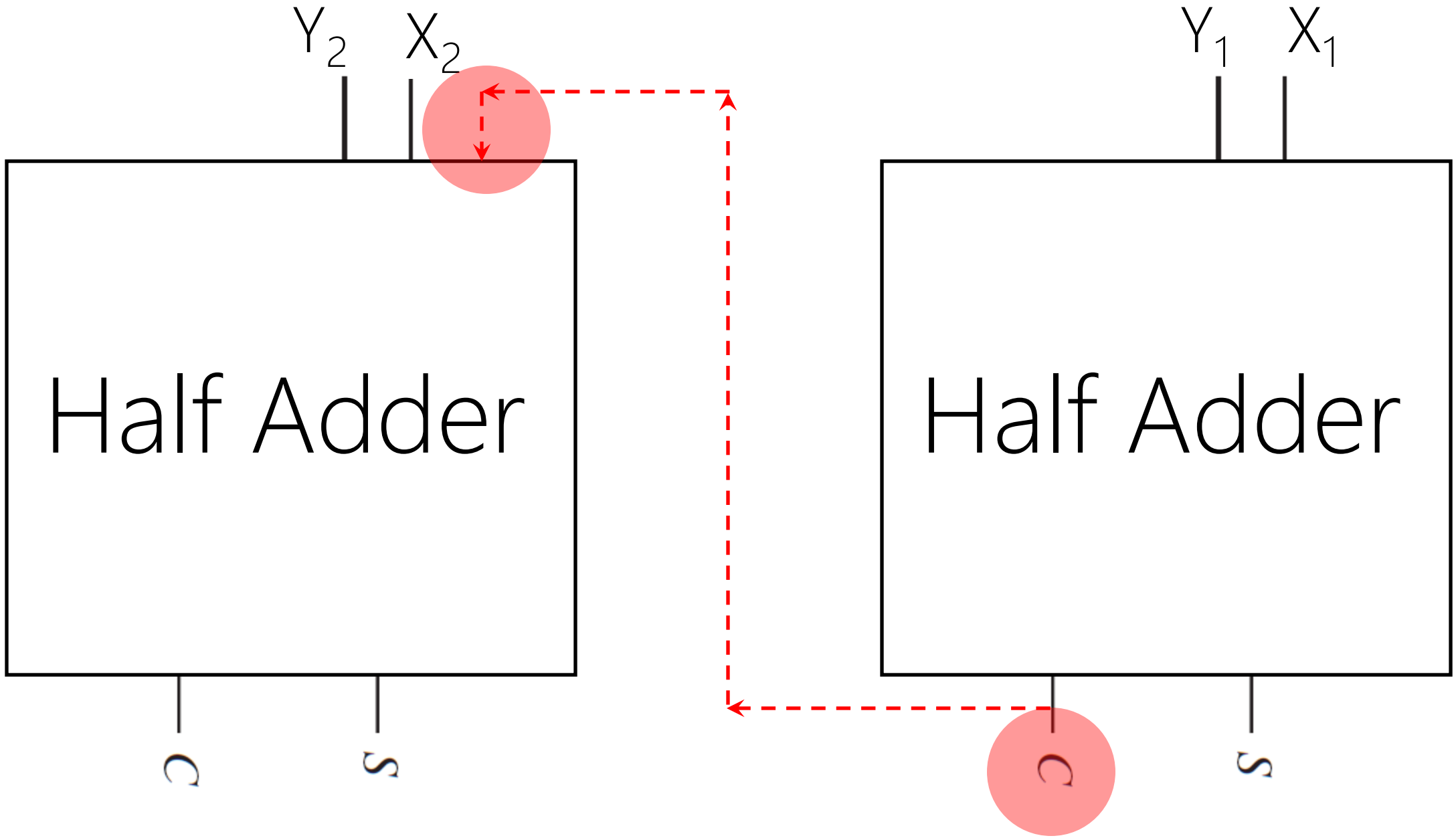

y
x
Half Adder
C
S

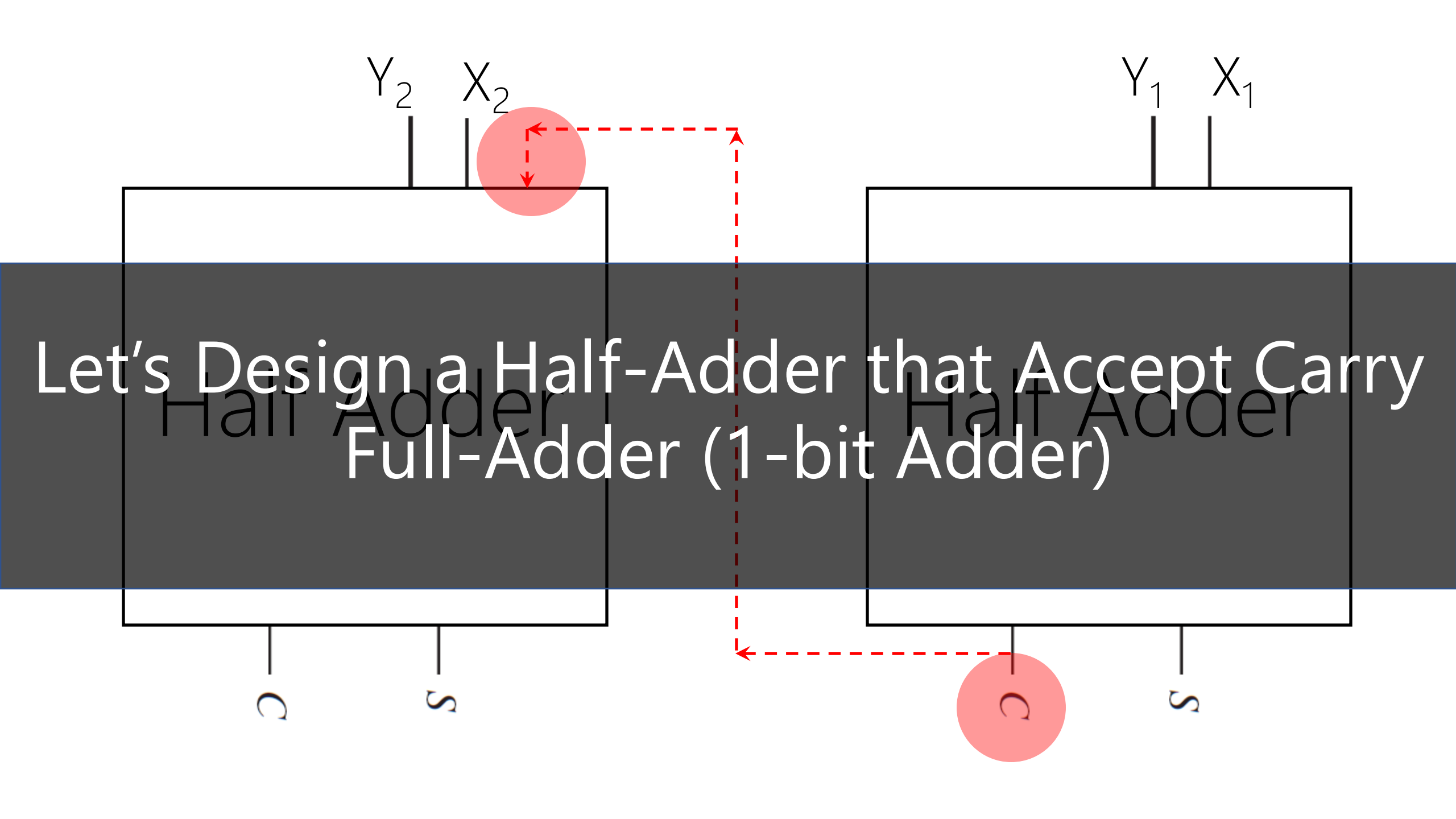

Y_2
X_2
Half Adder
C
S_2
Y_1
X_1
Half Adder
S_1


Y_2
X_2
Half Adder
C
S_2
Y_1
X_1
Half Adder
C
S_1

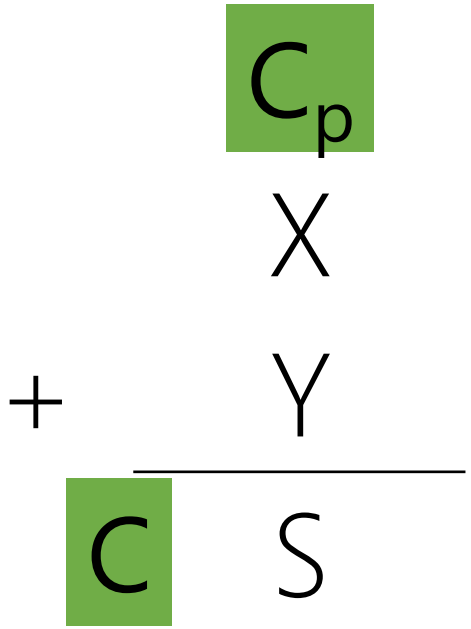
C=1

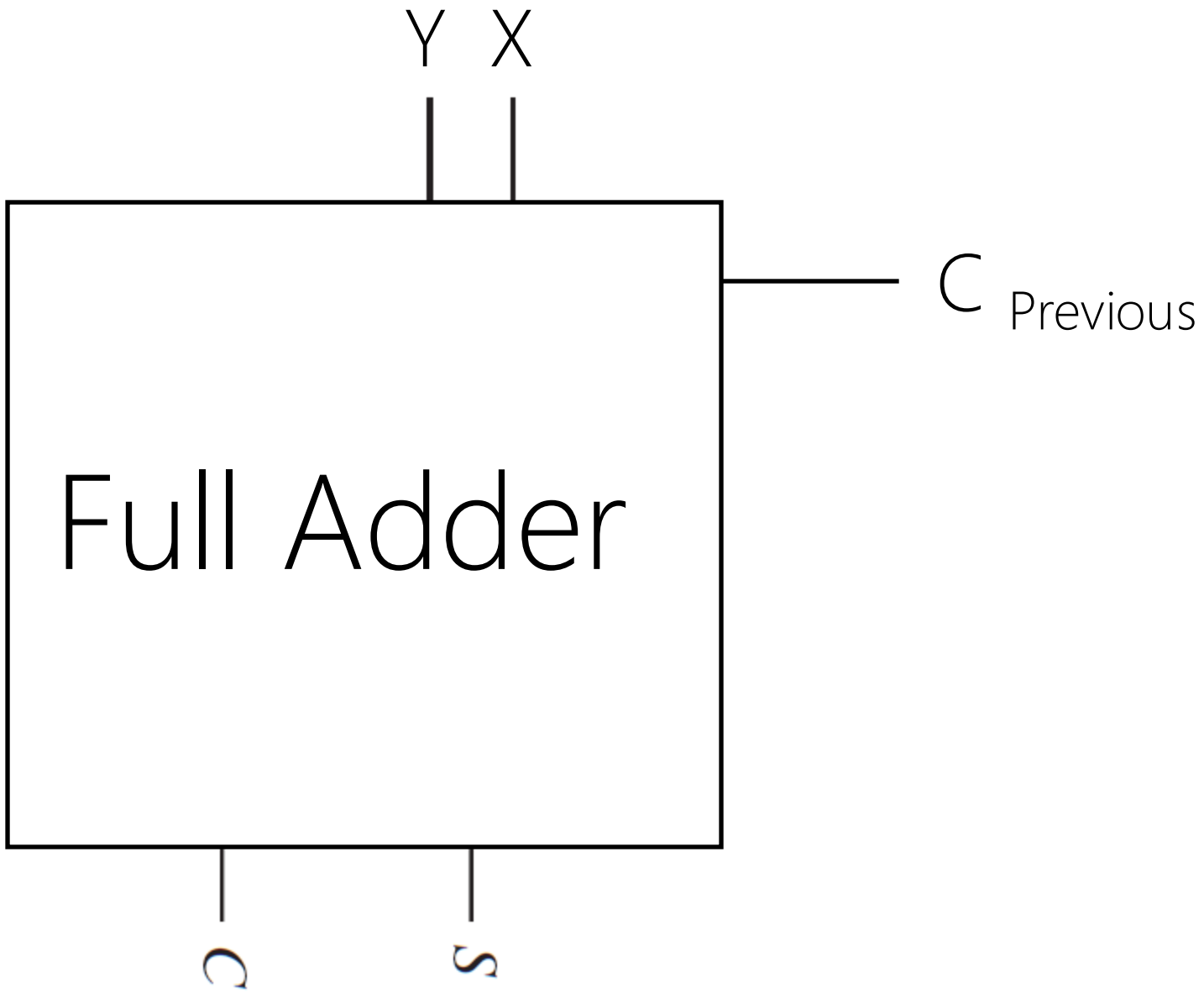
```
    0 1
+   0 1
-------
C=0 1 0
```


Y_2
X_2
Half Adder
C
S
Y_1
X_1
Half Adder
C
S



Let's Design a Half-Adder that Accept Carry
Full-Adder (1-bit Adder)


C_p
X
+ Y
C S


Y X
C Previous
Full Adder
C
S

Design a logic circuit that adds two binary digits (bit) and a carry bit.

C_p	Y	X	$C=\sum m(3,5,6,7)$	$S=\sum m(1,2,4,7)$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

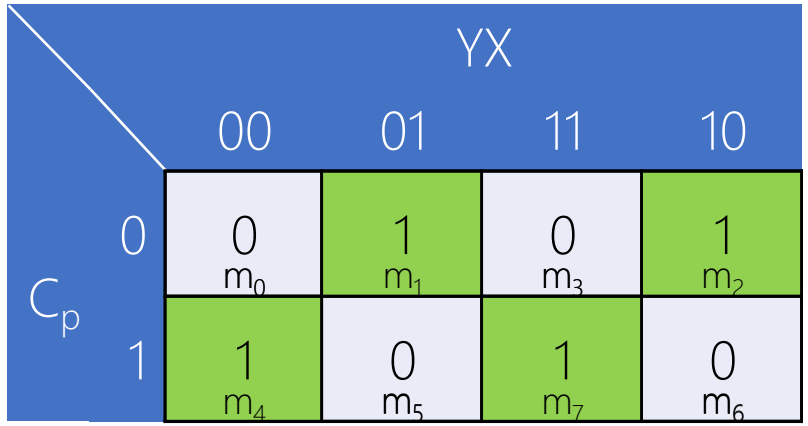
$$S=\sum m(1,2,4,7)$$

C_p \ YX	00	01	11	10
0	0 m_0	1 m_1	0 m_3	1 m_2
1	1 m_4	0 m_5	1 m_7	0 m_6

$$C=\sum m(3,5,6,7)$$

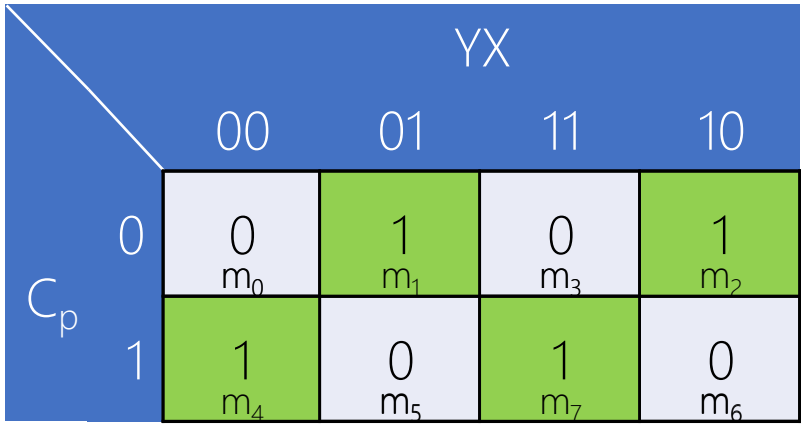
C_p \ YX	00	01	11	10
0	0 m_0	0 m_1	1 m_3	0 m_2
1	0 m_4	1 m_5	1 m_7	1 m_6

$$S=\sum m(1,2,4,7)$$



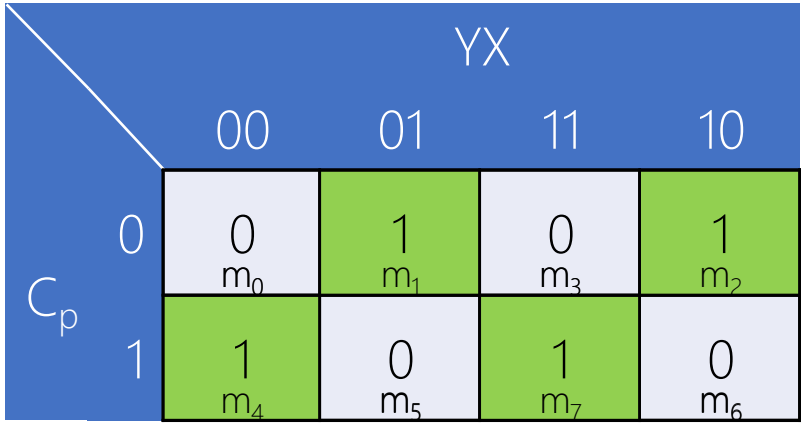
$$S=C'_pY'X+C'_pYX'+C_pY'X'+C_pYX$$

$$S=\sum m(1,2,4,7)$$



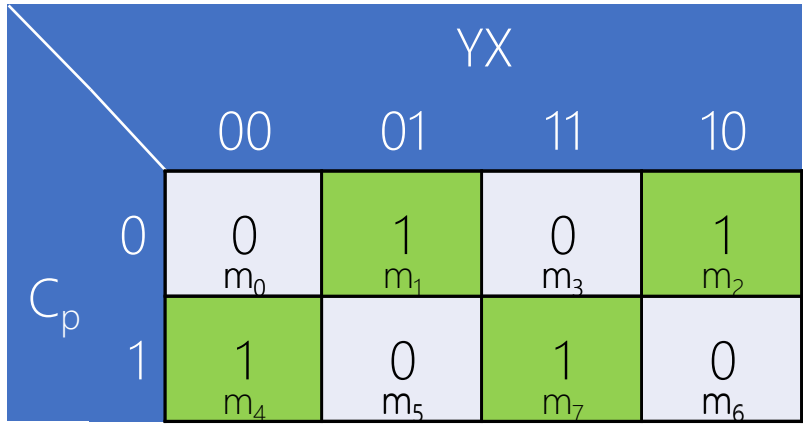
$$S=C'_pY'X+C'_pYX'+C_pY'X'+C_pYX$$
$$=C'_p(Y'X+YX')+C_p(Y'X'+YX)$$

$$S=\sum m(1,2,4,7)$$



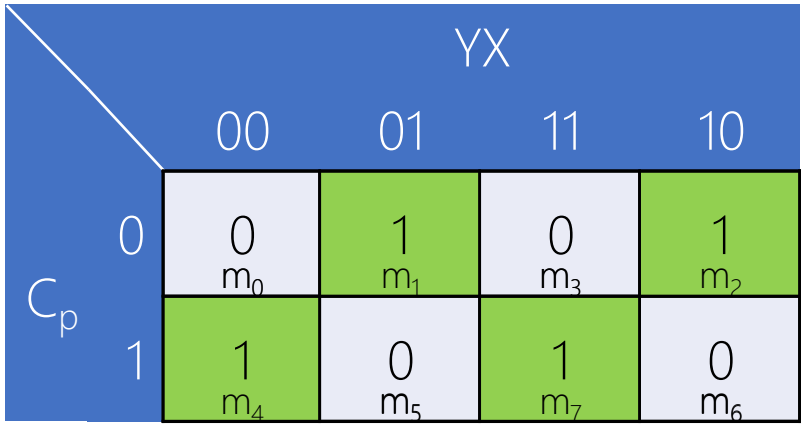
$$S=C'_pY'X+C'_pYX'+C_pY'X'+C_pYX$$
$$=C'_p(Y'X+YX')+C_p(Y'X'+YX)$$
$$=C'_p(X\oplus Y)+C_p(Y'X'+YX)$$

$$S=\sum m(1,2,4,7)$$



$$S=C'_pY'X+C'_pYX'+C_pY'X'+C_pYX$$
$$=C'_p(Y'X+YX')+C_p(Y'X'+YX)$$
$$=C'_p(X\oplus Y)+C_p(Y'X'+YX)$$
$$=C'_p(X\oplus Y)+C_p(X\odot Y)$$

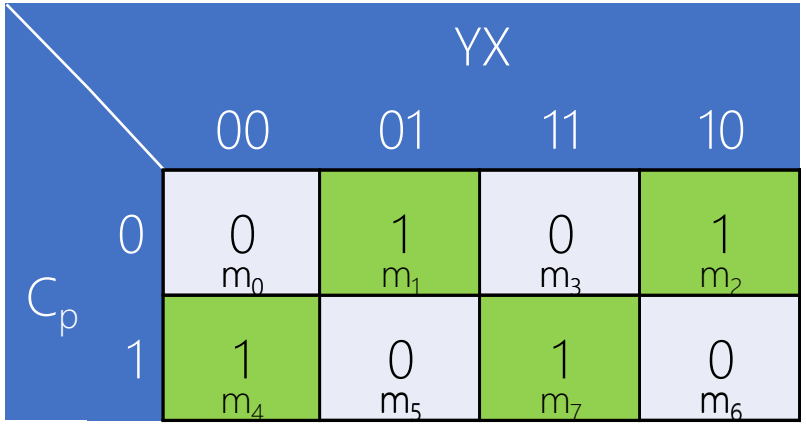
$$S=\sum m(1,2,4,7)$$



$$S=C'_pY'X+C'_pYX'+C_pY'X'+C_pYX$$
$$=C'_p(Y'X+YX')+C_p(Y'X'+YX)$$
$$=C'_p(X\oplus Y)+C_p(Y'X'+YX)$$
$$=C'_p(X\oplus Y)+C_p(X\odot Y)$$
$$=C'_p(X\oplus Y)+C_p(X\oplus Y)'$$

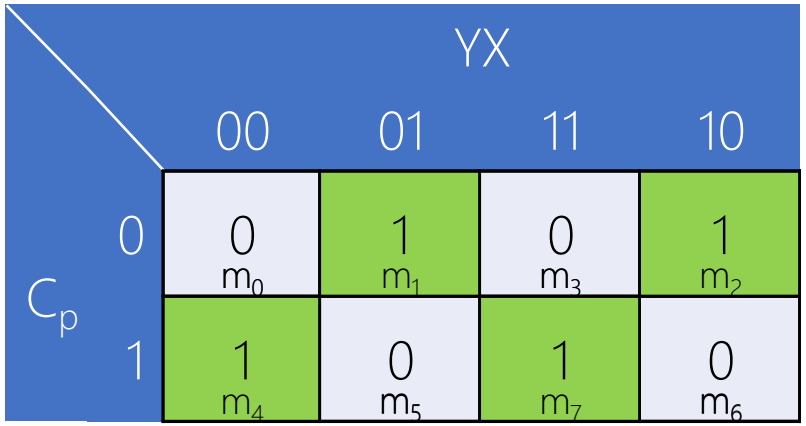
$$\begin{aligned}(X\oplus Y)' &= (Y'X+YX')'\\ &= (Y'X)'(YX')'\\ &= (Y+X')(Y'+X)\\ &= YY'+YX+X'Y'+X'X\\ &= 0+YX+X'Y'+0\\ &= YX+X'Y'\\ &= Y\odot X\end{aligned}$$

$$S=\sum m(1,2,4,7)$$



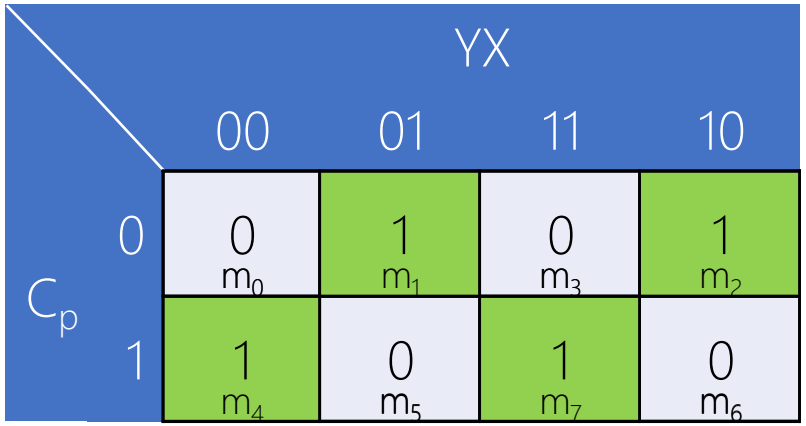
$$\begin{aligned}
S &= C'_pY'X + C'_pYX' + C_pY'X' + C_pYX \\
&= C'_p(Y'X + YX') + C_p(Y'X' + YX) \\
&= C'_p(X \oplus Y) + C_p(Y'X' + YX) \\
&= C'_p(X \oplus Y) + C_p(X \odot Y) \\
&= C'_p(X \oplus Y) + C_p(X \oplus Y)' \\
&= C'_p\alpha + C_p\alpha'
\end{aligned}$$

$$S=\sum m(1,2,4,7)$$



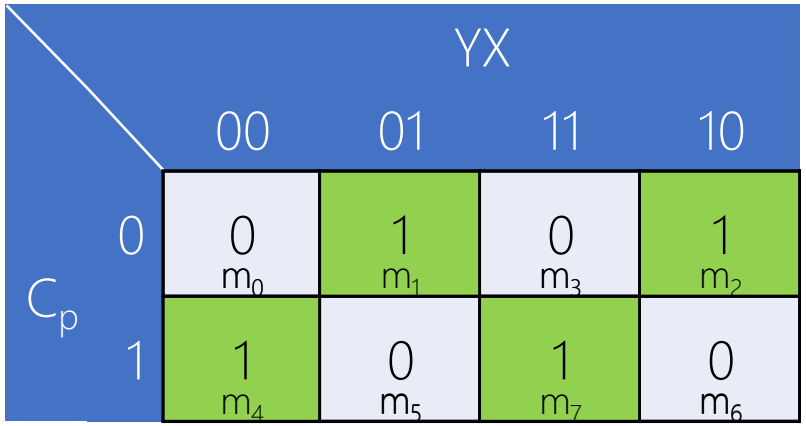
$$\begin{aligned}
S &= C'_pY'X + C'_pYX' + C_pY'X' + C_pYX \\
&= C'_p(Y'X + YX') + C_p(Y'X' + YX) \\
&= C'_p(X \oplus Y) + C_p(Y'X' + YX) \\
&= C'_p(X \oplus Y) + C_p(X \odot Y) \\
&= C'_p(X \oplus Y) + C_p(X \oplus Y)' \\
&= C'_p\alpha + C_p\alpha' \\
&= C_p \oplus \alpha
\end{aligned}$$

$$S=\sum m(1,2,4,7)$$



$$\begin{aligned}
S&=C'_pY'X+C'_pYX'+C_pY'X'+C_pYX\\
&=C'_p(Y'X+YX')+C_p(Y'X'+YX)\\
&=C'_p(X\oplus Y)+C_p(Y'X'+YX)\\
&=C'_p(X\oplus Y)+C_p(X\odot Y)\\
&=C'_p(X\oplus Y)+C_p(X\oplus Y)'\\
&=C'_p\alpha+C_p\alpha'\\
&=C_p\oplus\alpha\\
&=C_p\oplus(X\oplus Y)
\end{aligned}$$

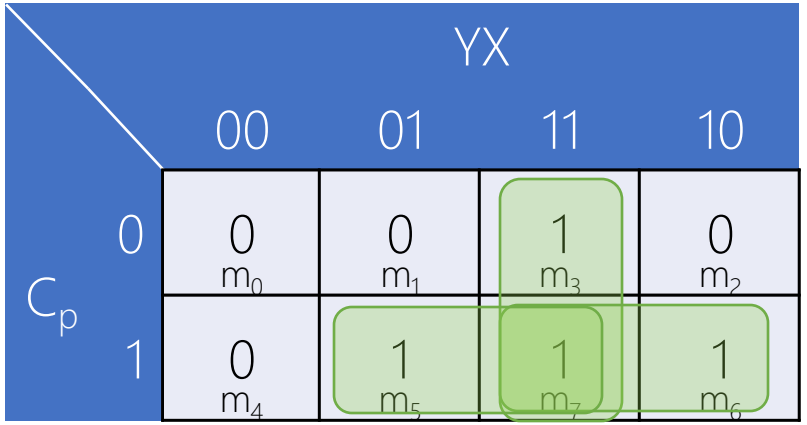
$$S = \sum m(1,2,4,7)$$
$$= C_p \oplus (X \oplus Y)$$

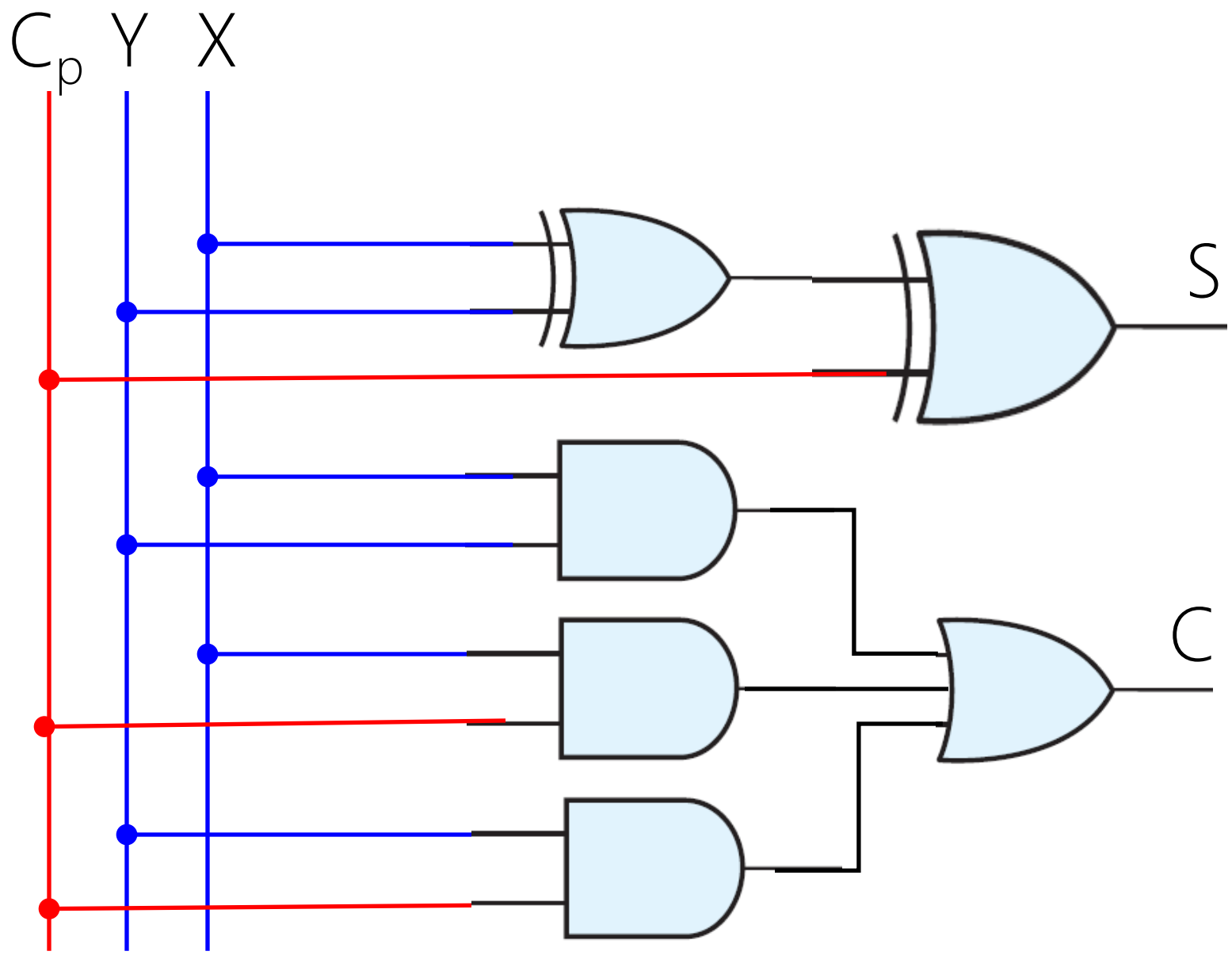


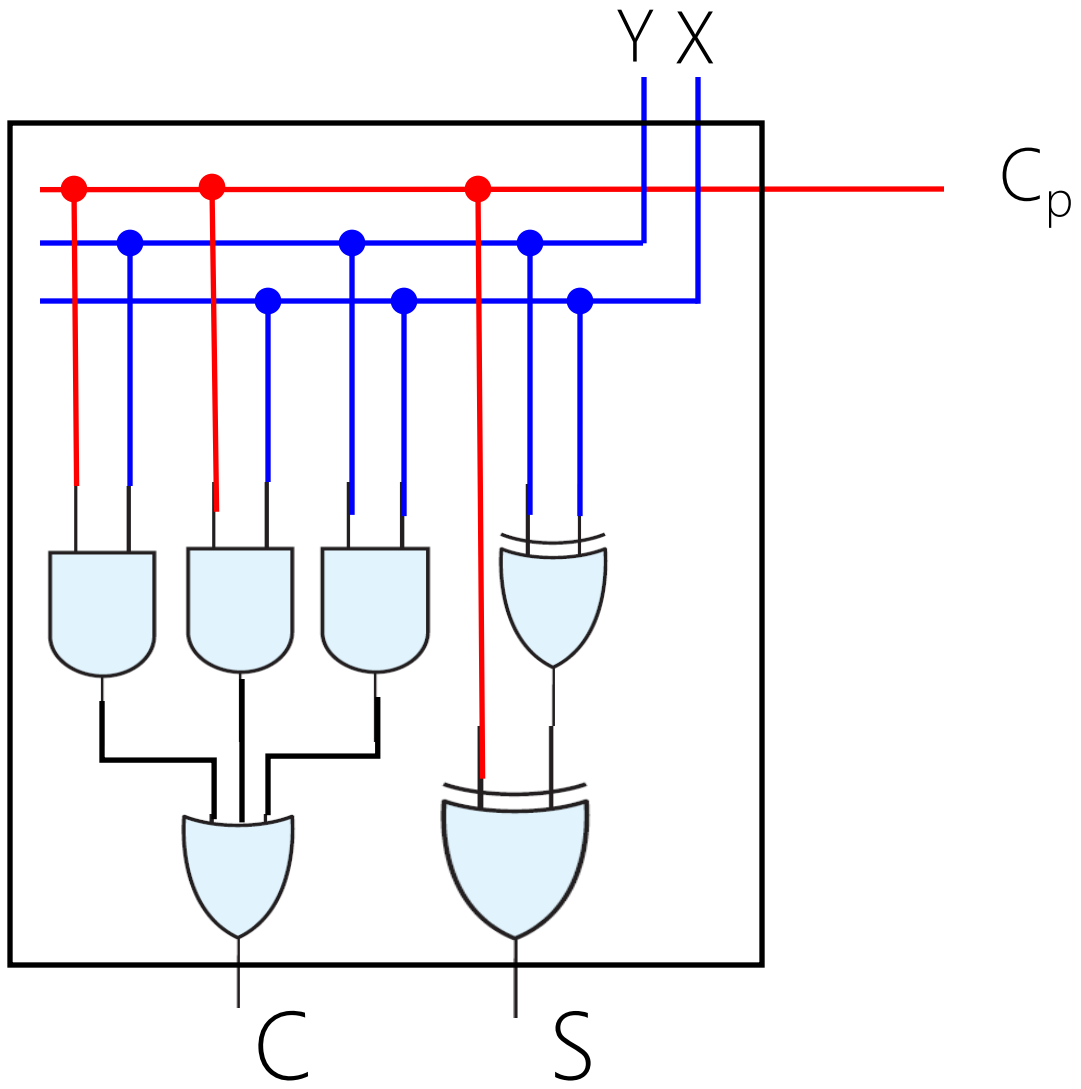
$\oplus$ is associative, we can drop ( ). But let's keep them!

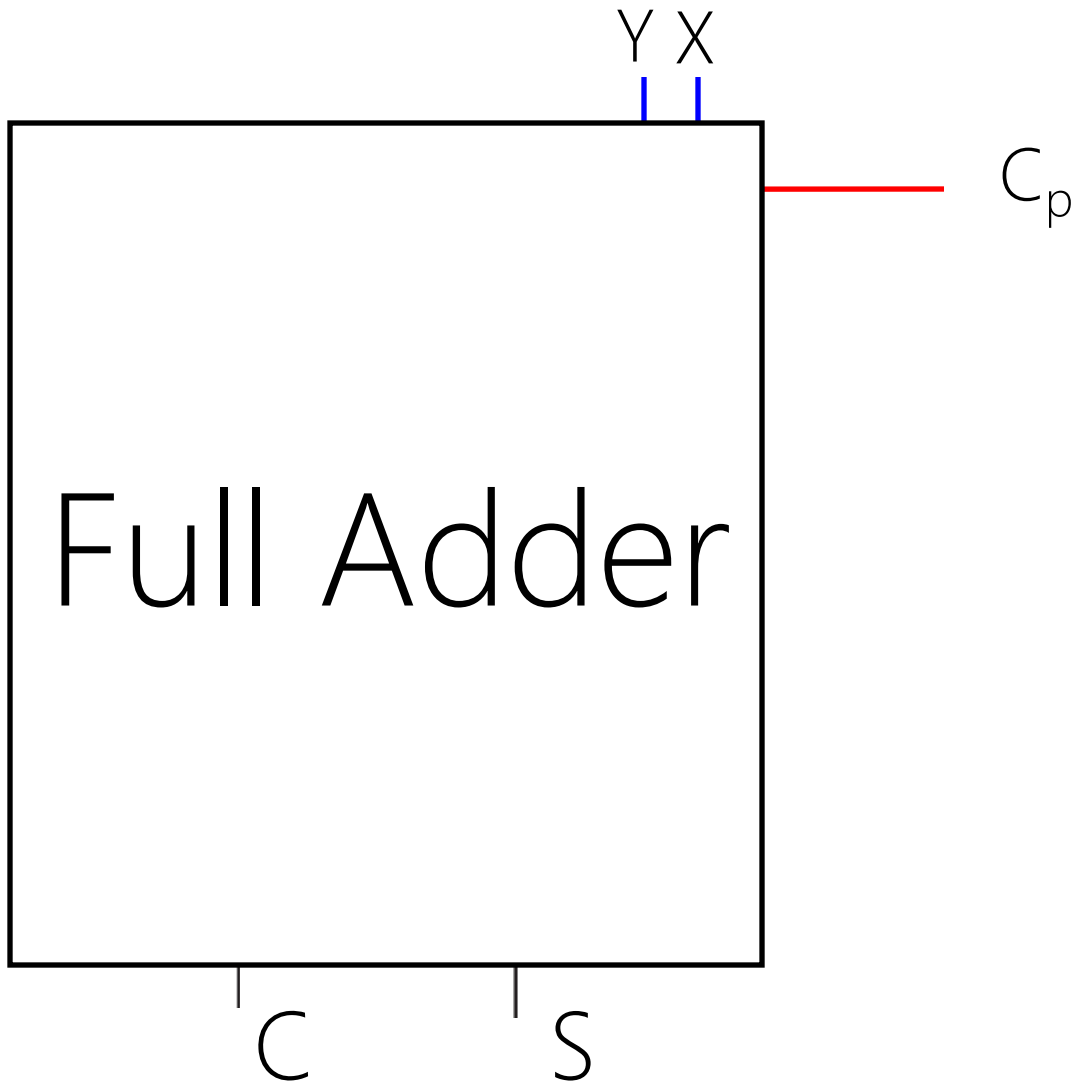
$$C = \sum m(3,5,6,7)$$

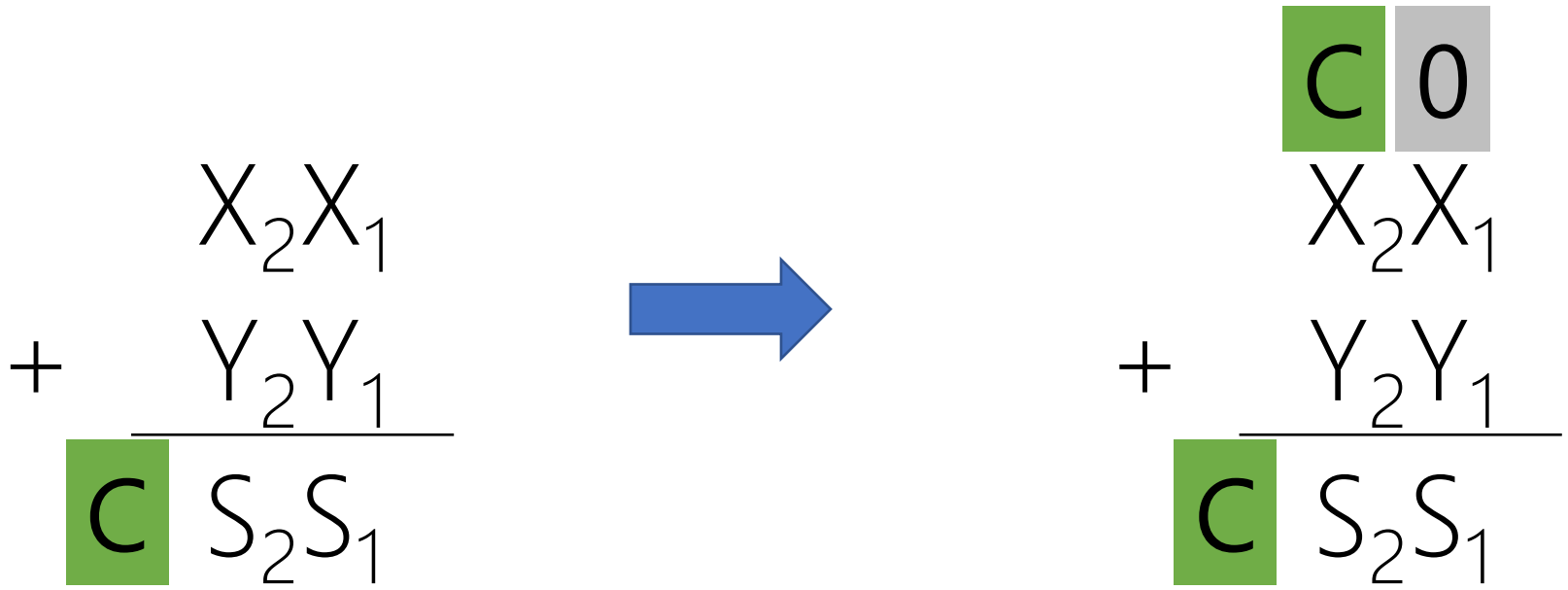
$$= YX + C_p X + C_p Y$$

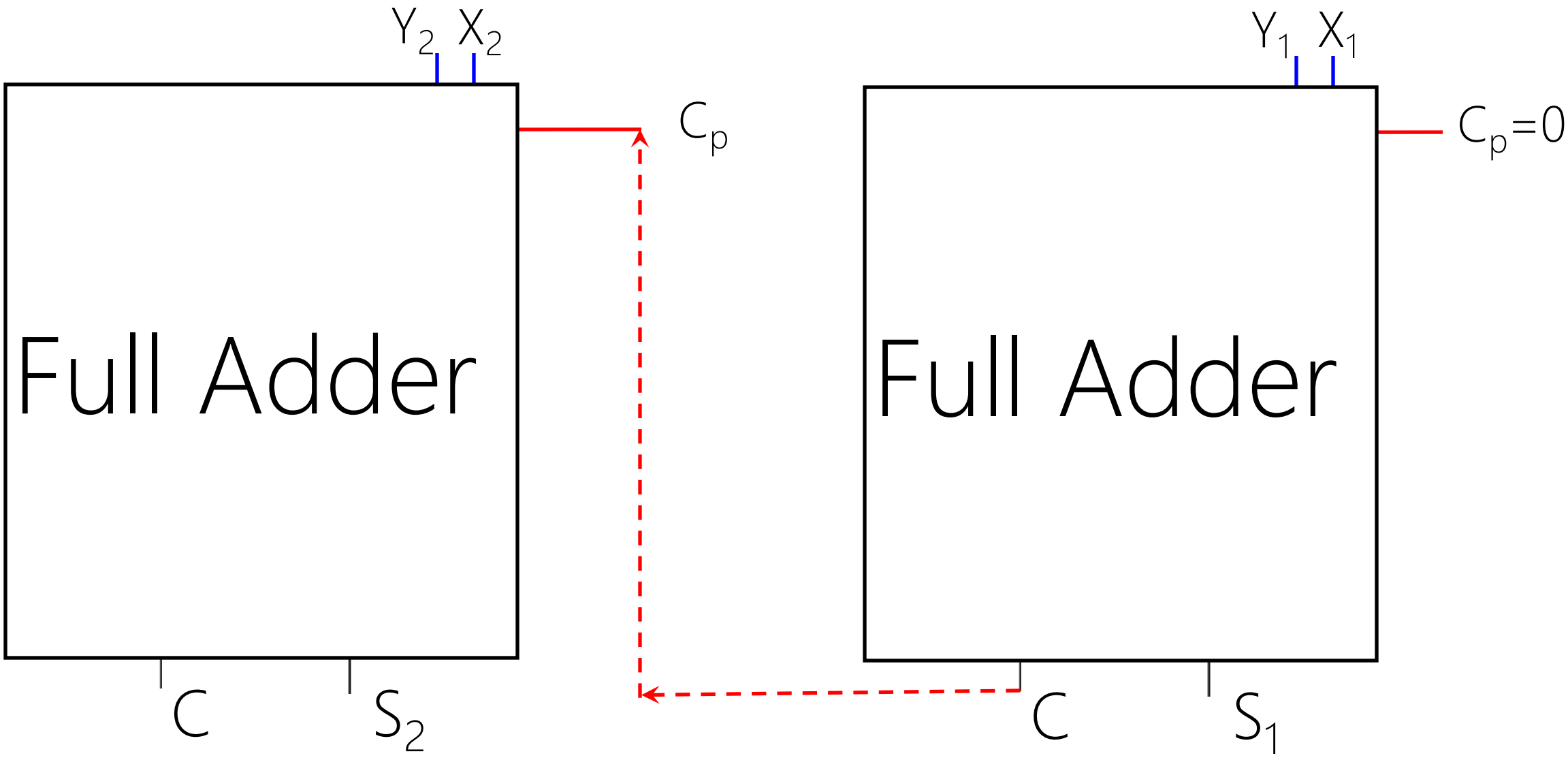



C_p Y X
S
C

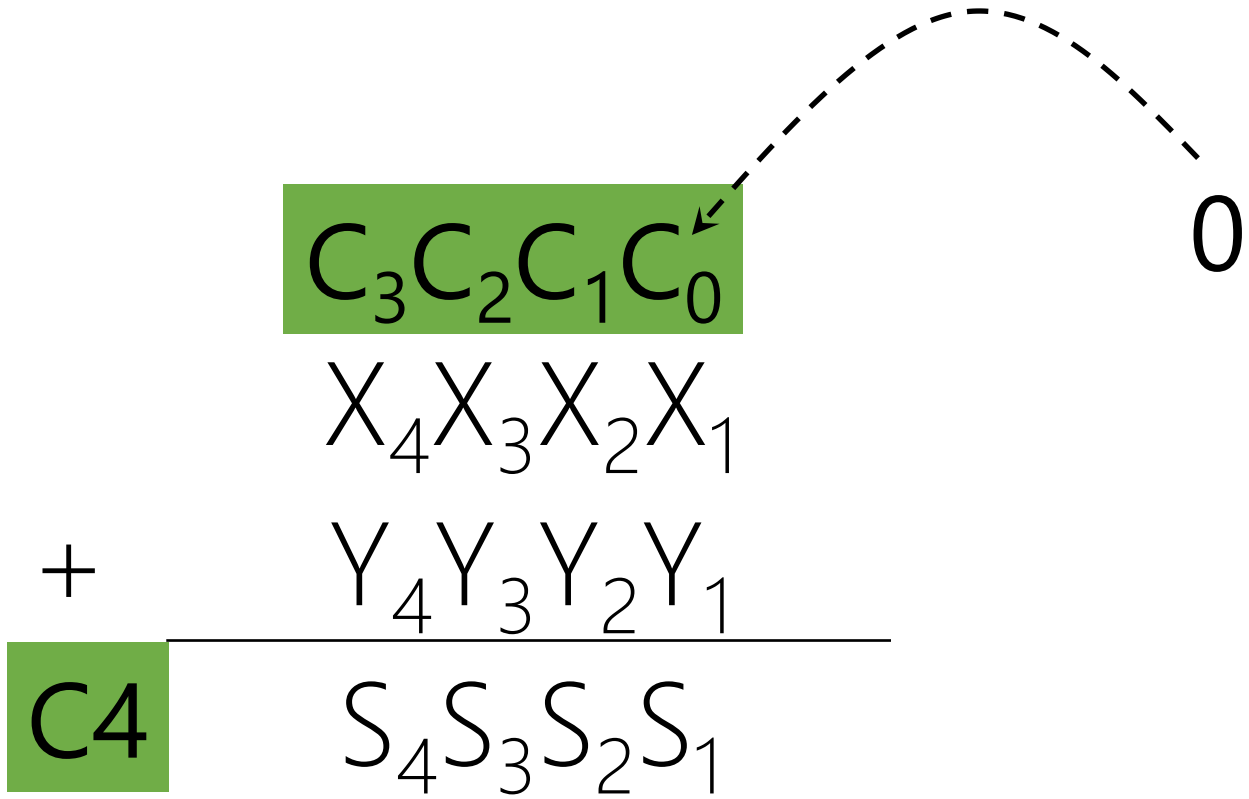

Y X
C_p
C
S

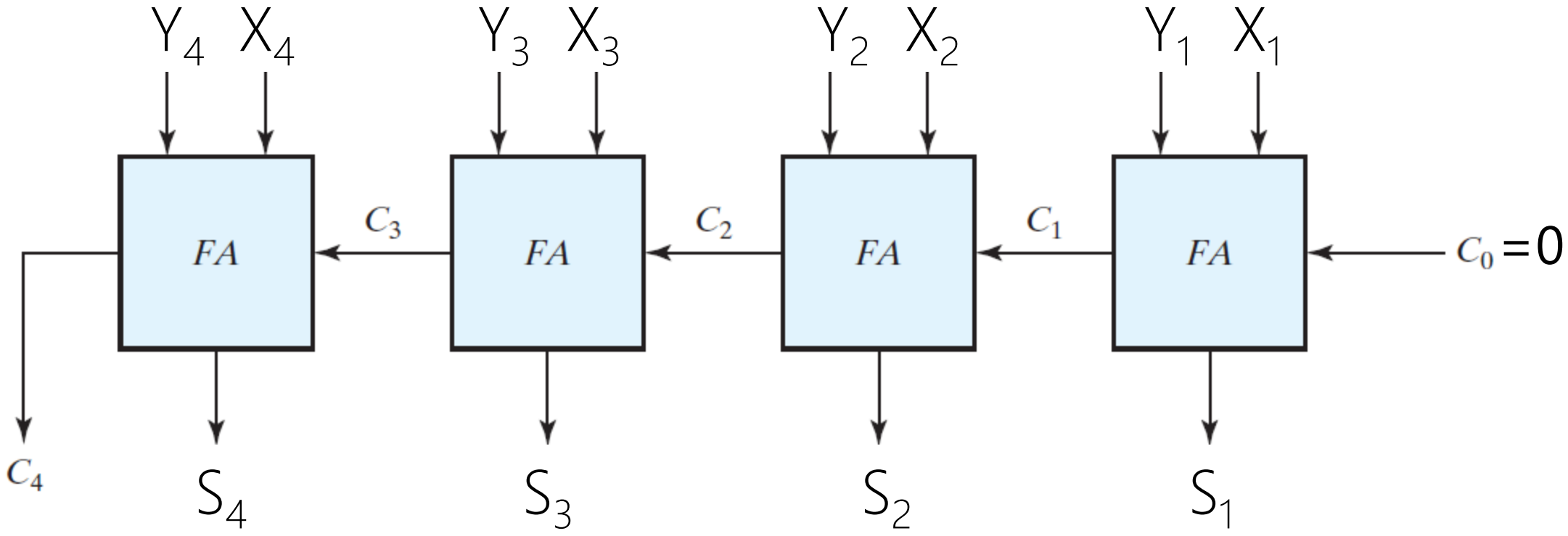

Y X
C_p
Full Adder
C
S

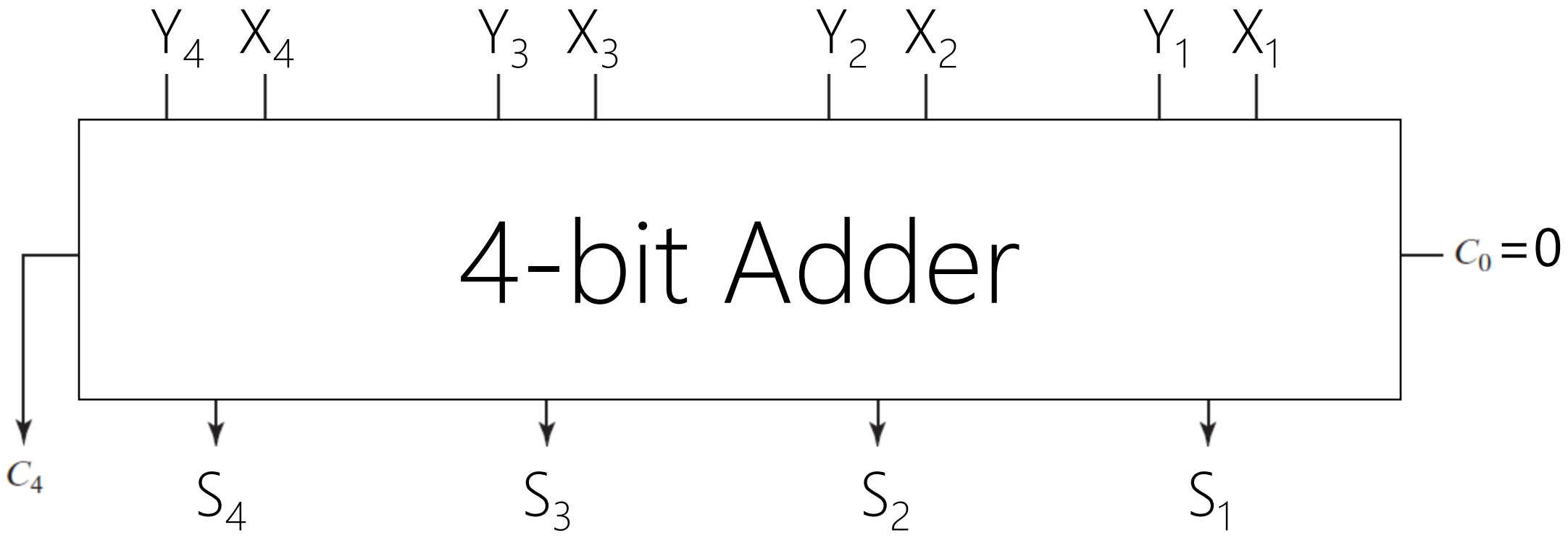

X_2X_1
$+ \quad Y_2Y_1$
$C \; S_2S_1$
C 0
X_2X_1
$+ \quad Y_2Y_1$
$C \; S_2S_1$

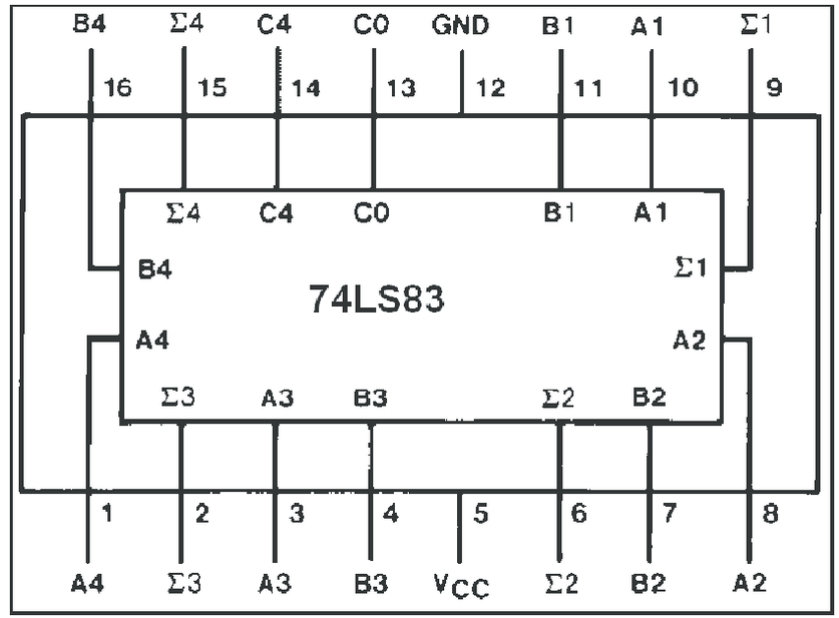

Y2 X2
Cp
Full Adder
C
S2
Y1 X1
Cp=0
Full Adder
C
S1

Design a logic circuit that adds two binary numbers!

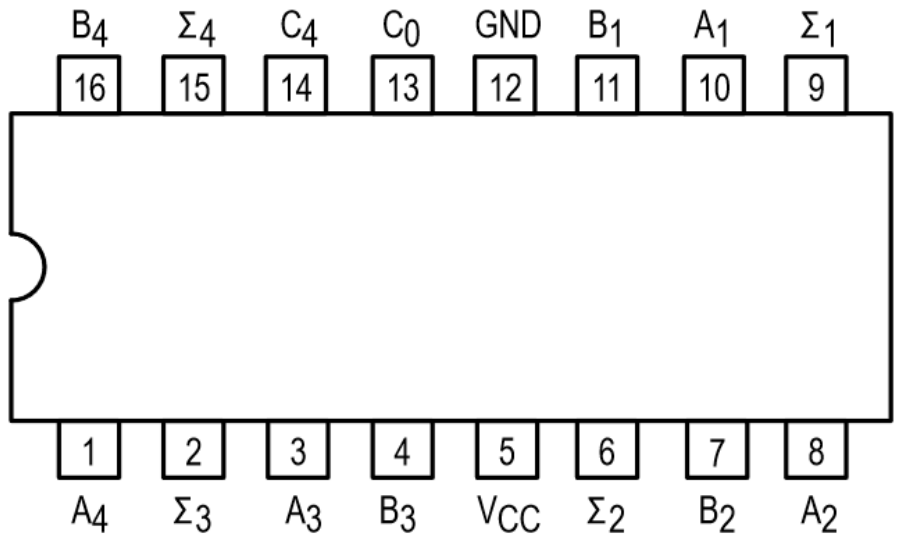

$C_3C_2C_1C_0$
0
$X_4X_3X_2X_1$
+
$Y_4Y_3Y_2Y_1$
C4
$S_4S_3S_2S_1$


Y4 X4
Y3 X3
Y2 X2
Y1 X1
FA
FA
FA
FA
C3
C2
C1
C0 = 0
C4
S4
S3
S2
S1


Y4 X4
Y3 X3
Y2 X2
Y1 X1
4-bit Adder
C0 =0
C4
S4
S3
S2
S1

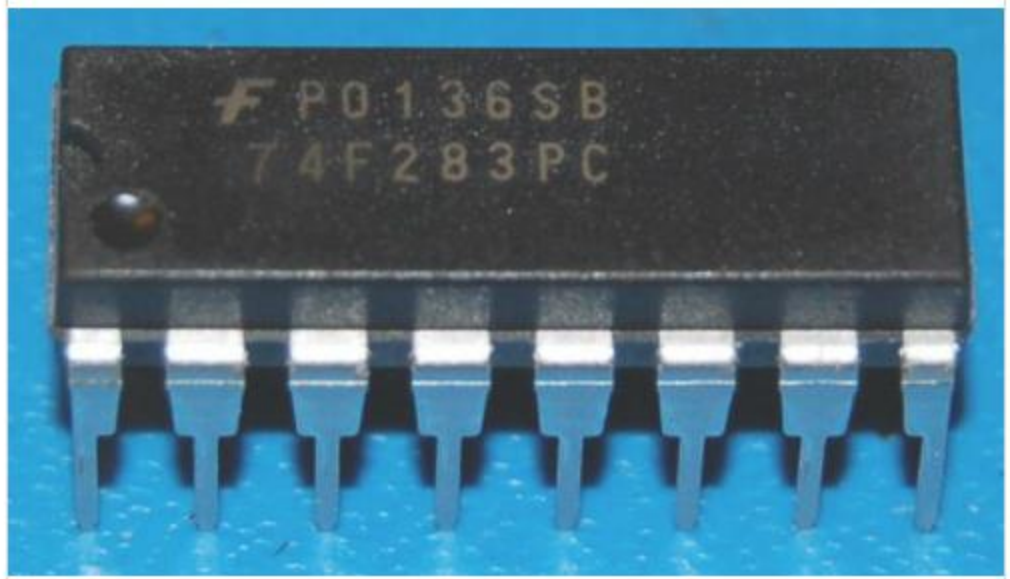


74LS83 pinout



Vcc	5.5V max, 5V Typical
$A_1 - A_4$	Operand A Inputs
$B_1 - B_4$	Operand B Inputs
C_0	Carry Input
$\Sigma_1 - \Sigma_4$	Sum Outputs (Note b)
C_4	Carry Output (Note b)

74283 - 74F283N 4-Bit Binary Full Adder w/ Fast Carry, DIP-16



C $6.55

+ C $4.89 Shipping

Get it by **Tue, Nov 10 - Tue, Nov 17** from Havre-aux-Maisons, QC, Canada

- **New** condition
- 30 day returns - Buyer pays return shipping |

Return policy

Read seller's description

See details

ebay MONEY BACK GUARANTEE

Qty: 1

Buy It Now

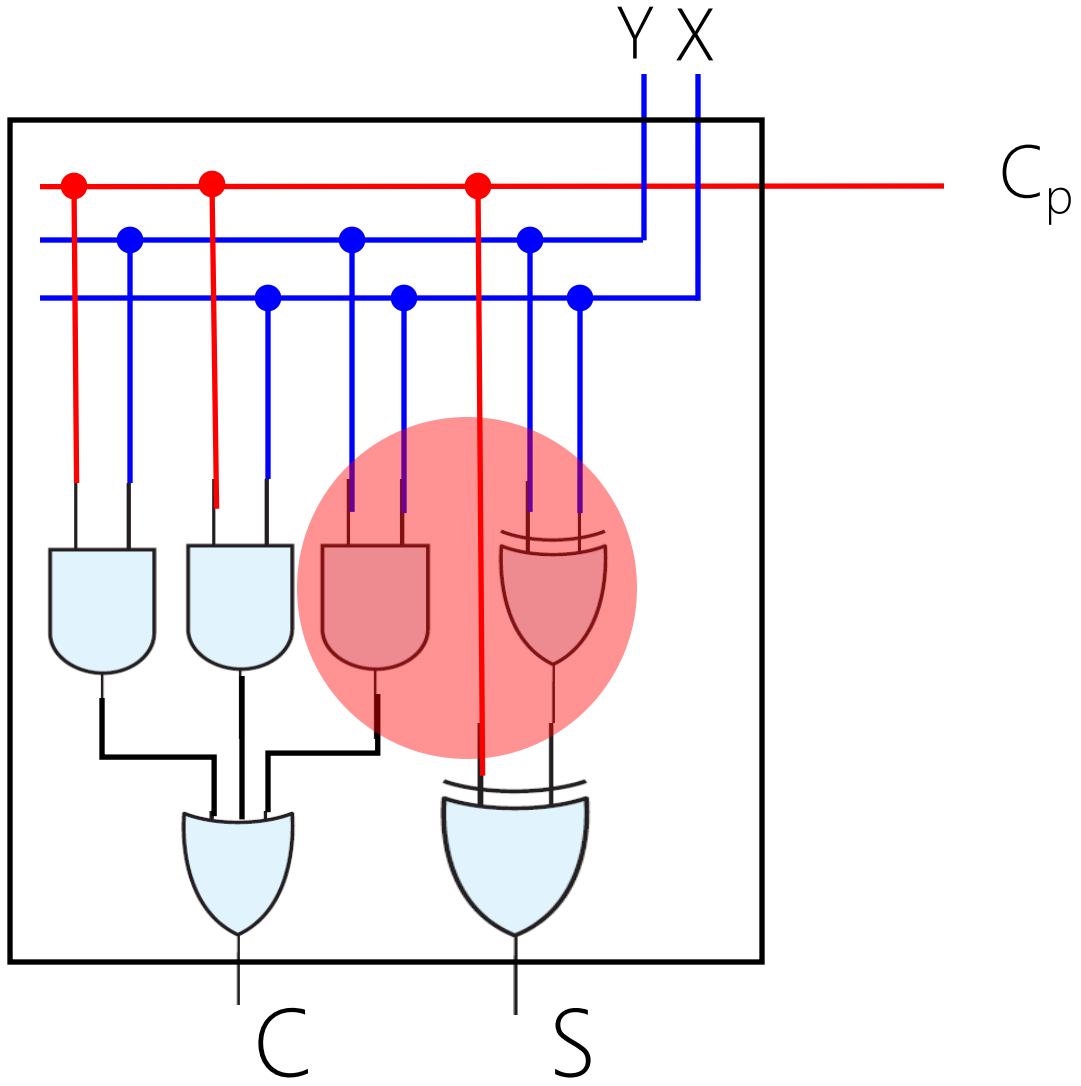
Add to cart

Watch

Sold by
vedge23 (3476)
100.0% Positive feedback
Contact seller

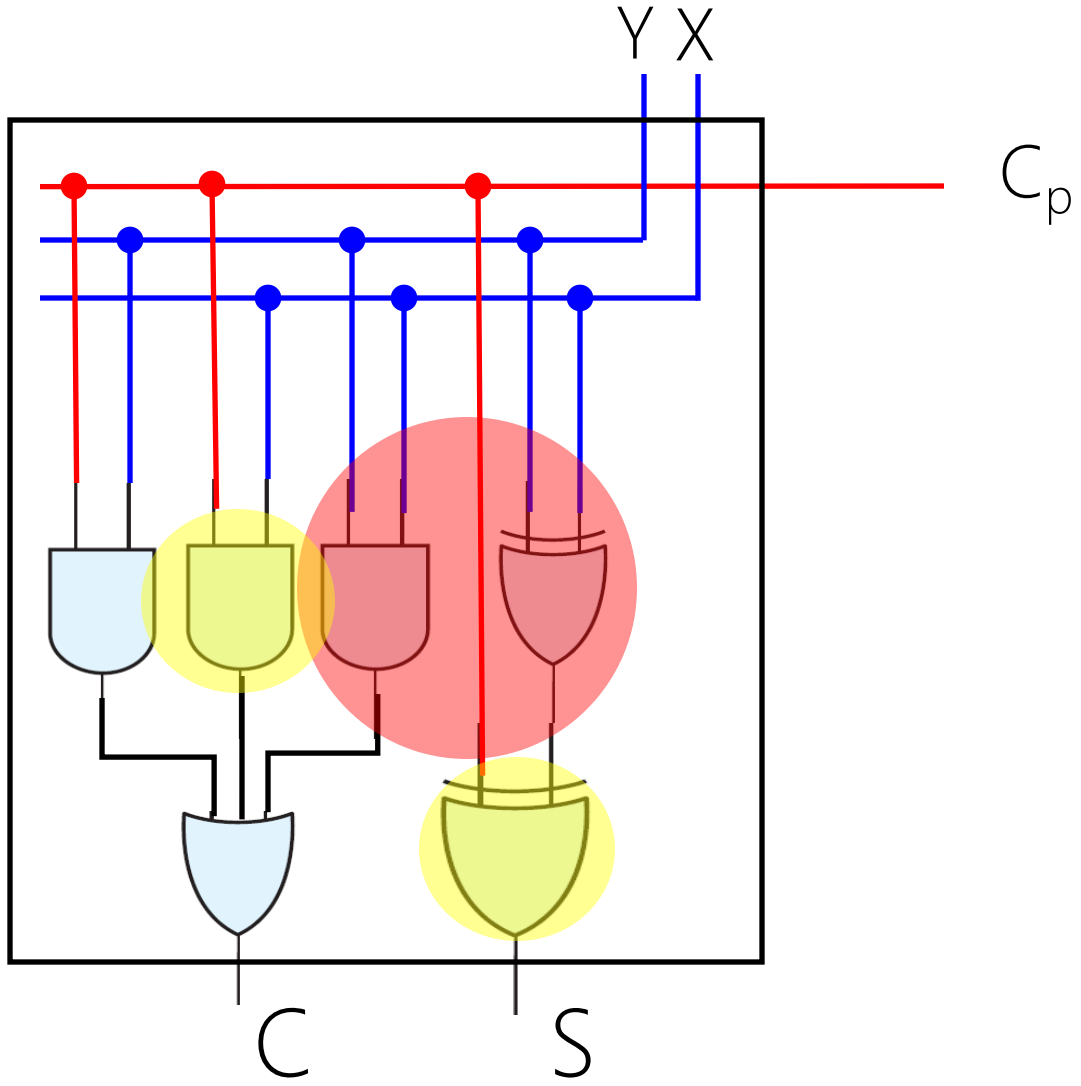
More on Full-Adder

Full Adder =? Half Adder + …



Lecture Assignment

Full Adder =? Half Adder + …

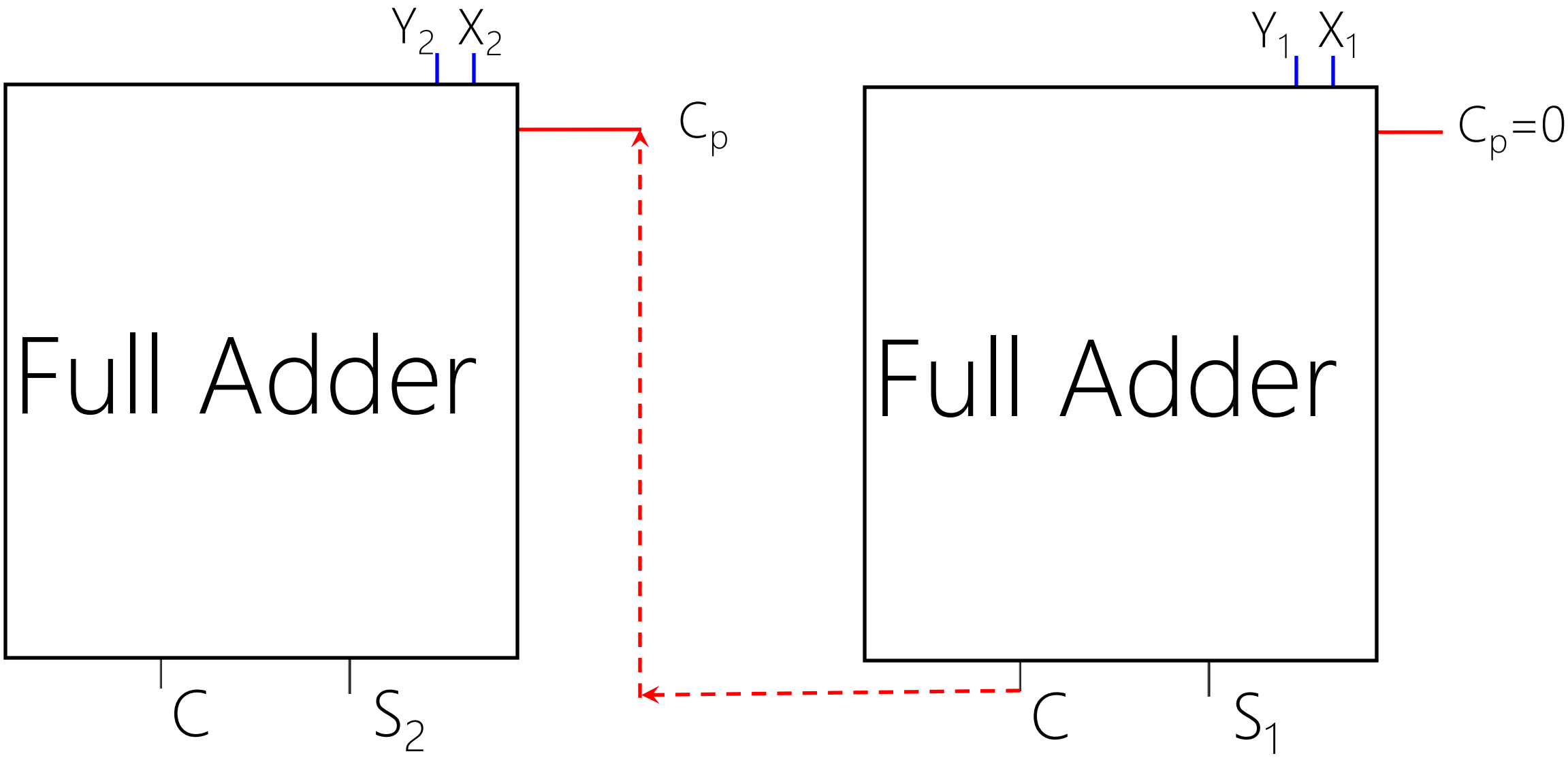


Lecture Assignment

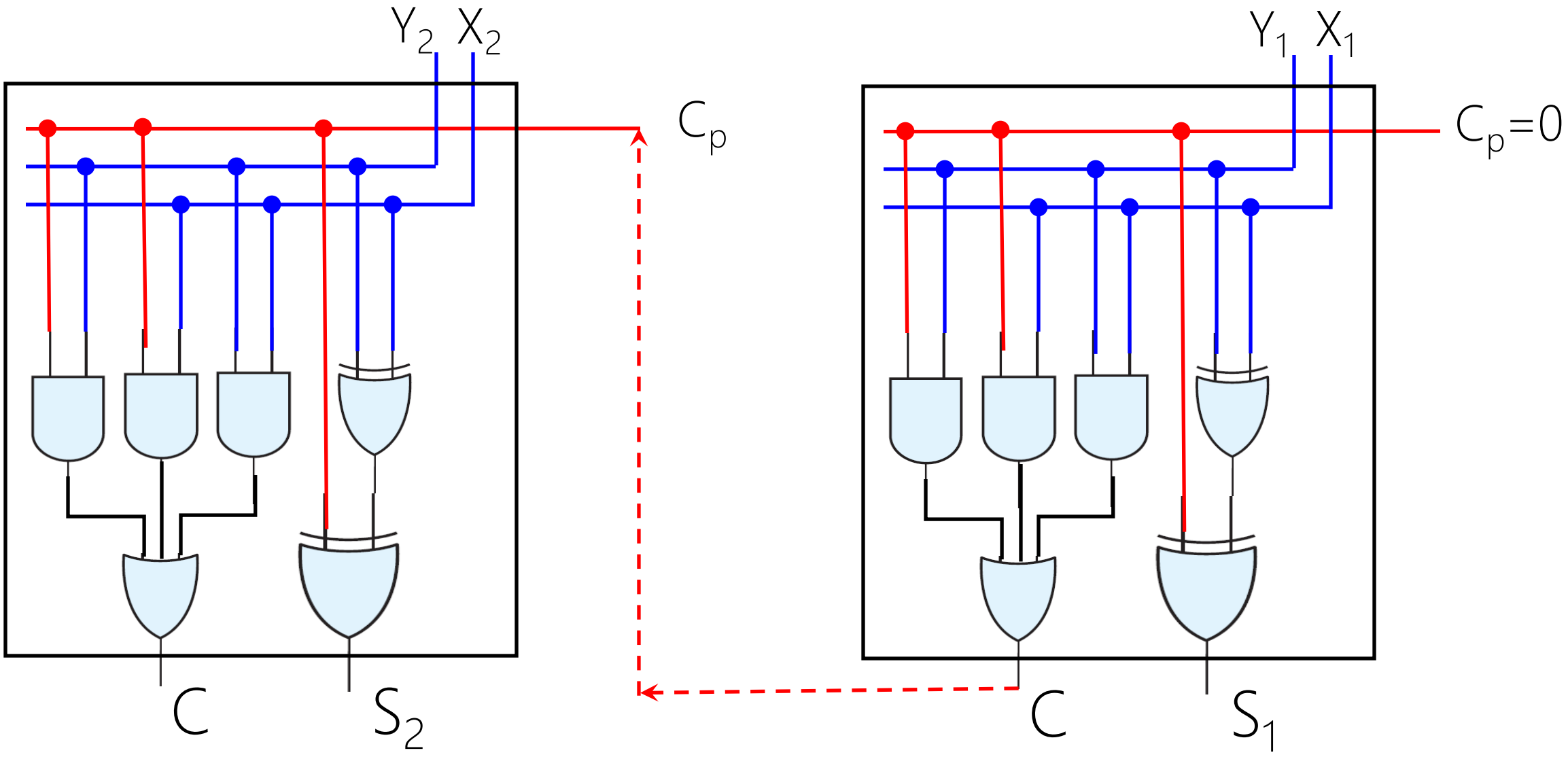
Full Adder ∝ 2 Half Adder

At Home!

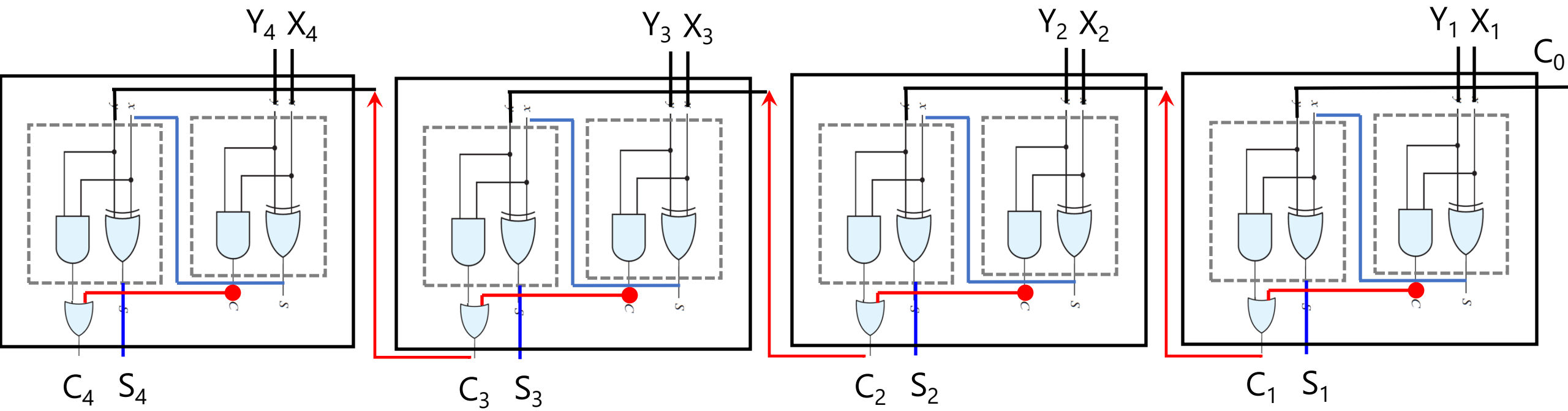
Carry Propagation


Y2 X2
Cp
Full Adder
C
S2
Y1 X1
Cp=0
Full Adder
C
S1

If gate delay is Δt, how long does it take to see the $S=S_2S_1$?



Lecture Assignment



If gate delay is Δt, how long does it take to see the $S=S_4S_3S_2S_1$?

At Home!

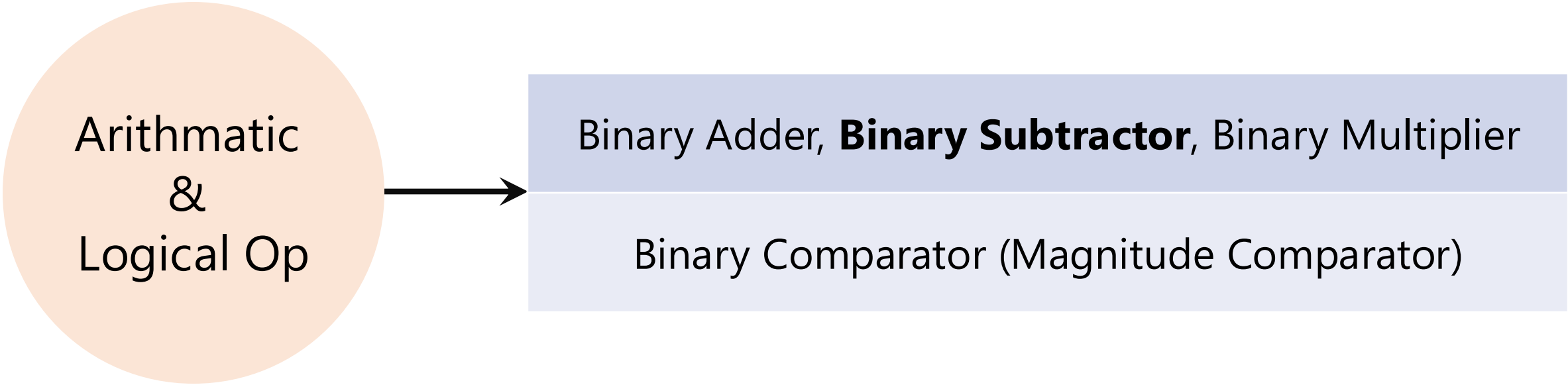
Carry Lookahead

$C_{1:n}$ → Constant Delay

Book Page 138-141

Full Adder

Does it matter we have signed or unsigned binary numbers?
Justify your answer.


Arithmatic
&
Logical Op
Binary Adder, **Binary Subtractor**, Binary Multiplier
Binary Comparator (Magnitude Comparator)

Binary Subtractor

Half-Subtractor ⟶ Full-Subtractor ⟶ n-bit Full-Subtractor

Lecture Assignments

Binary Subtractor

Signed-2's-Complement

X – Y

Subtraction in Signed-2's-Complement

X + 2's-comp(Y)

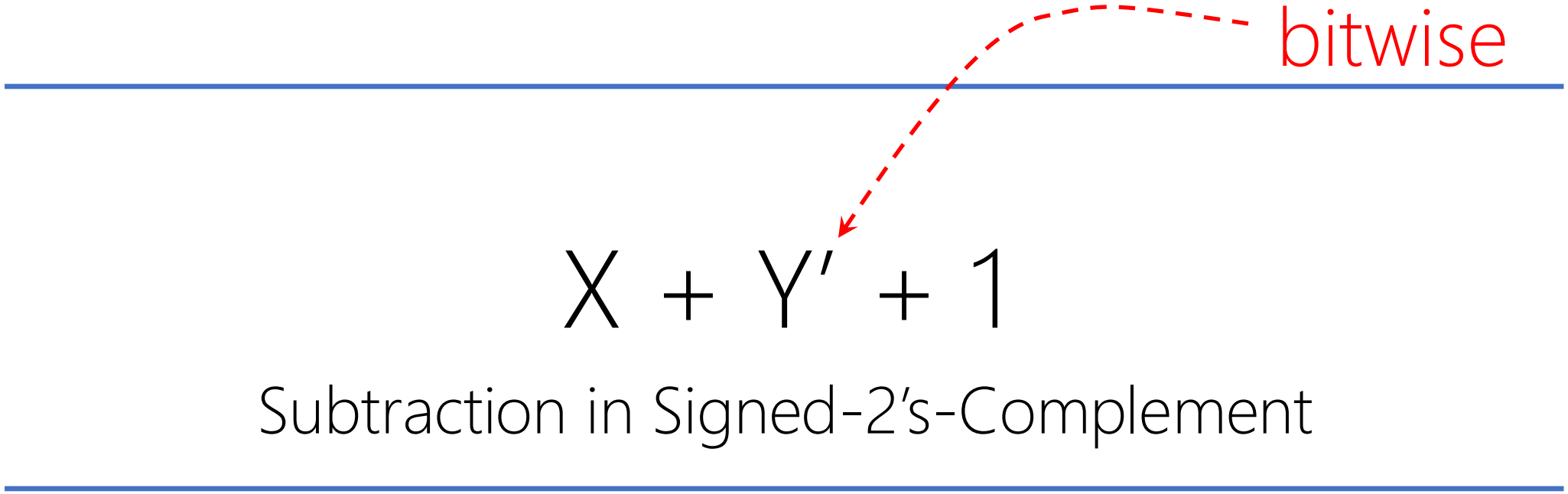
Subtraction in Signed-2's-Complement

X + 1's-comp(Y) + 1

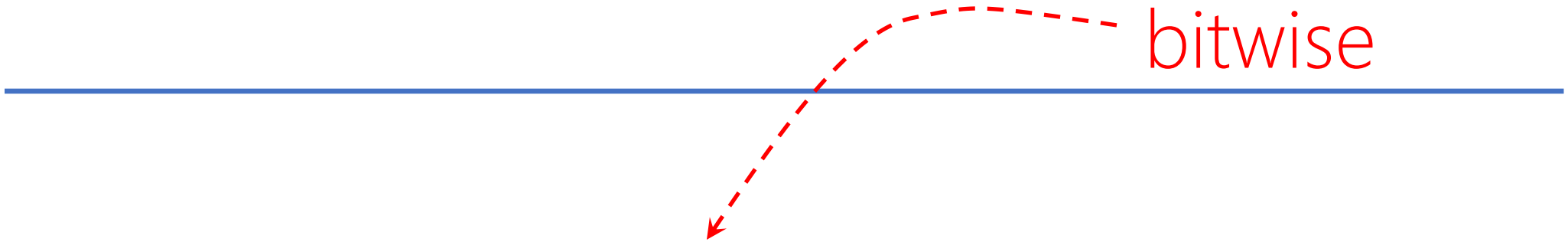
Subtraction in Signed-2's-Complement

$X_n X_{n-1} \ldots X_2 X_1 - Y_n Y_{n-1} \ldots Y_2 Y_1$

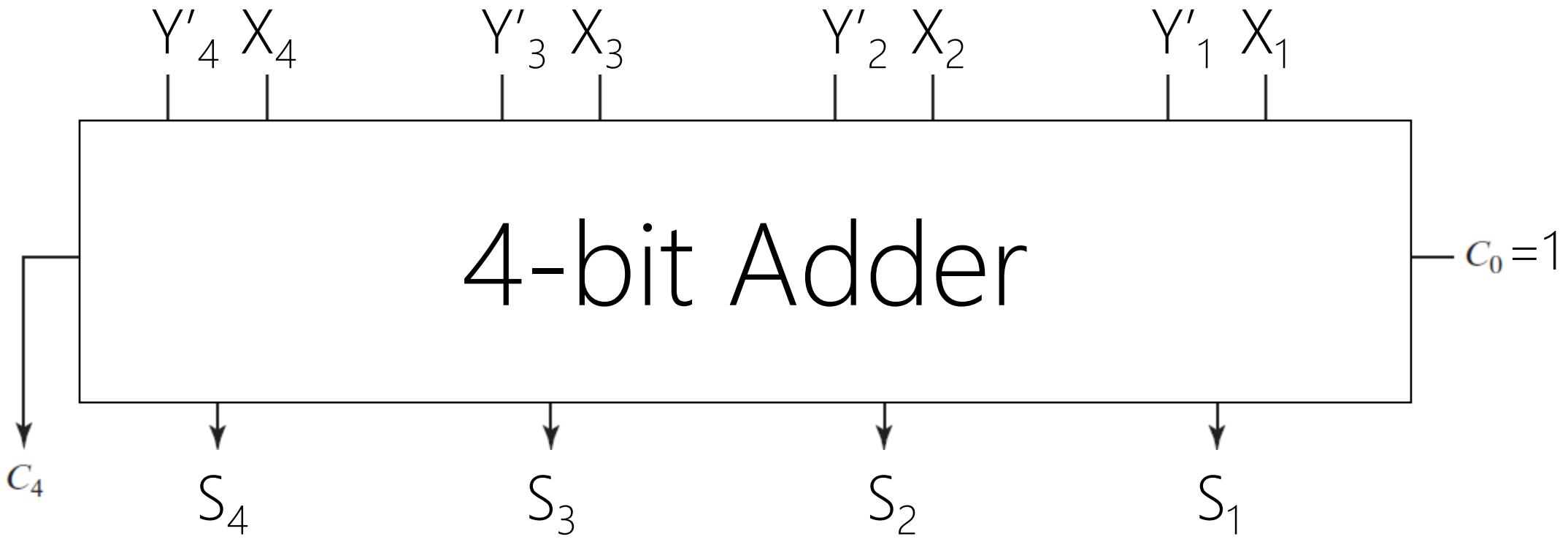
Subtraction in Signed-2's-Complement

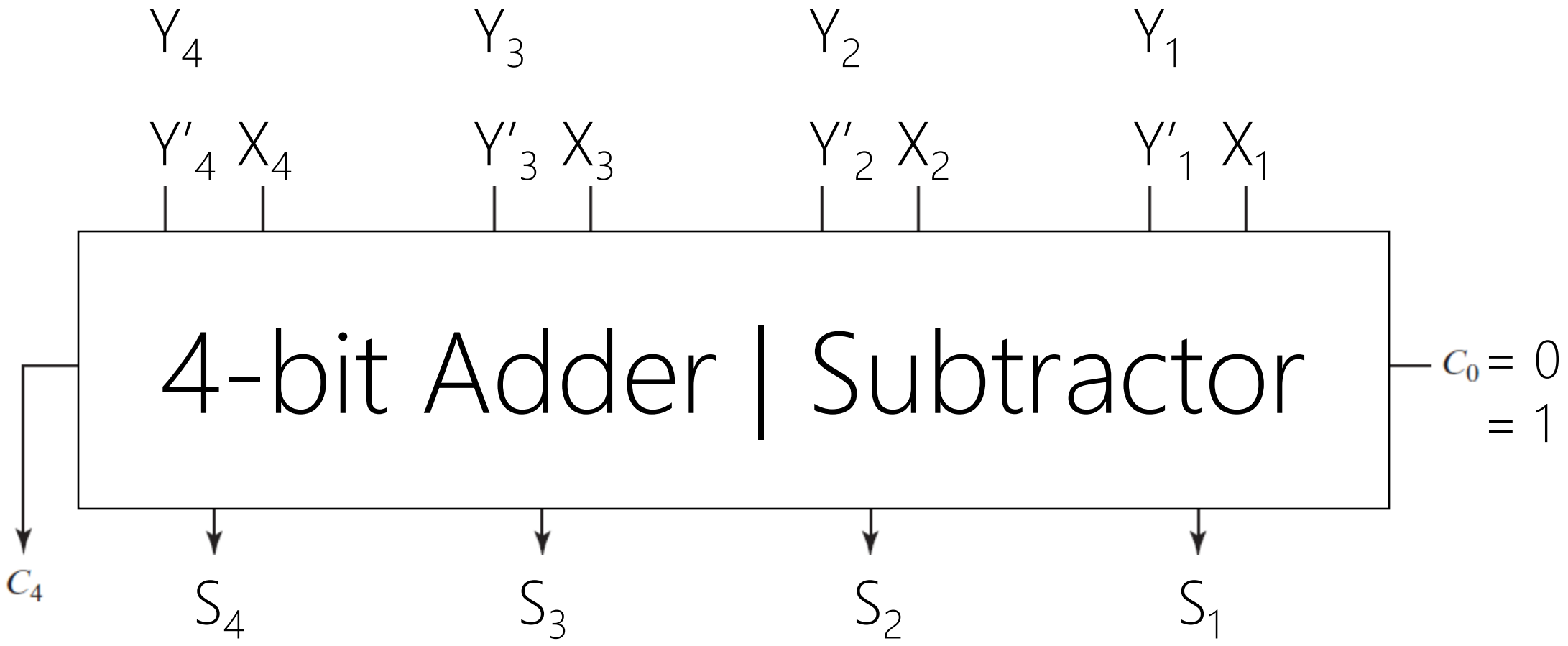


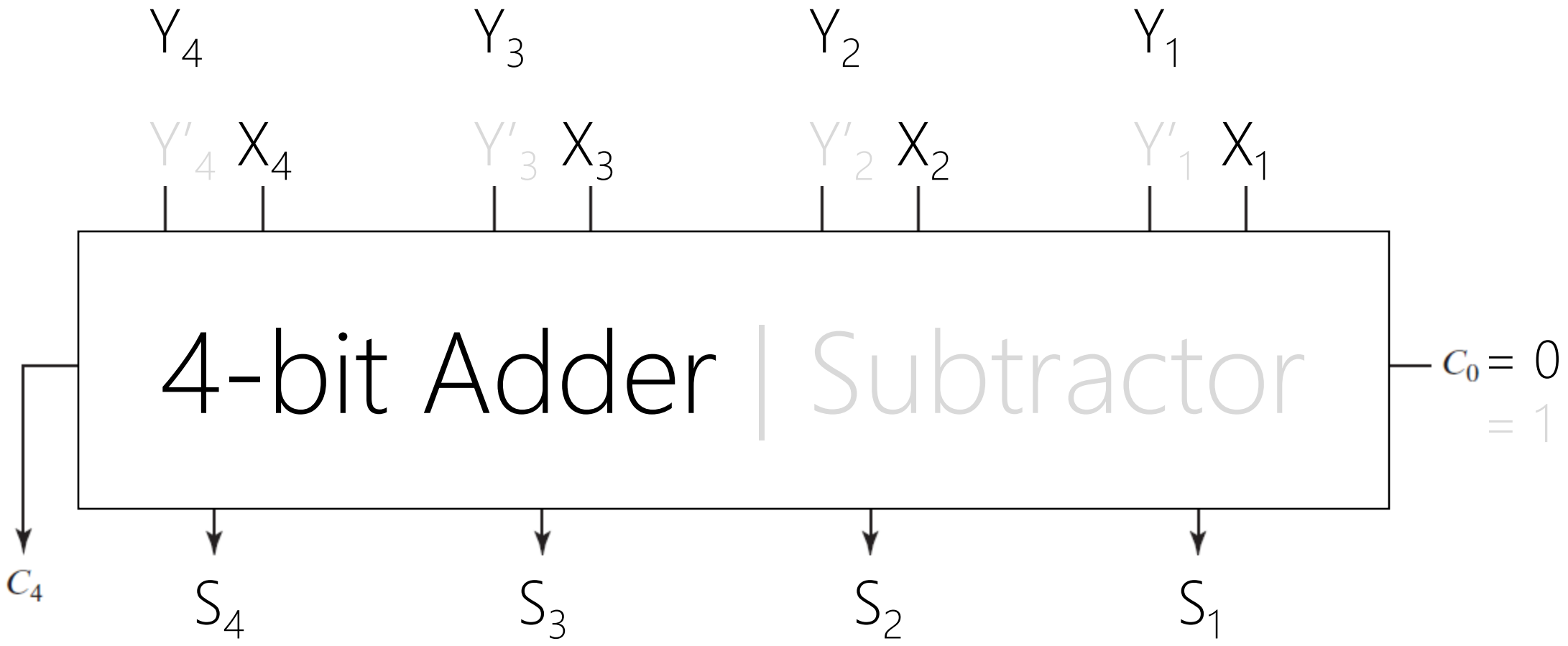
Subtraction in Signed-2's-Complement

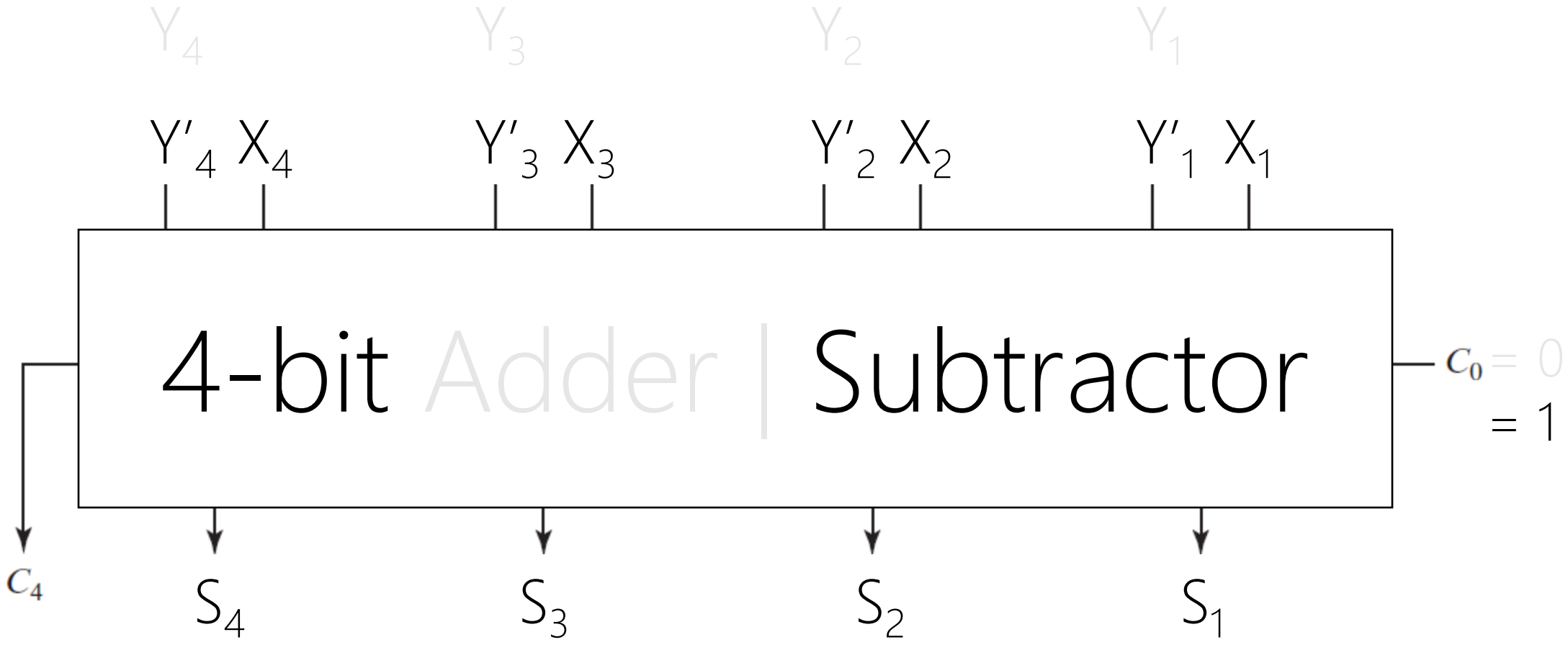


Subtraction in Signed-2's-Complement


Y′4 X4
Y′3 X3
Y′2 X2
Y′1 X1
4-bit Adder
C0 =1
C4
S4
S3
S2
S1

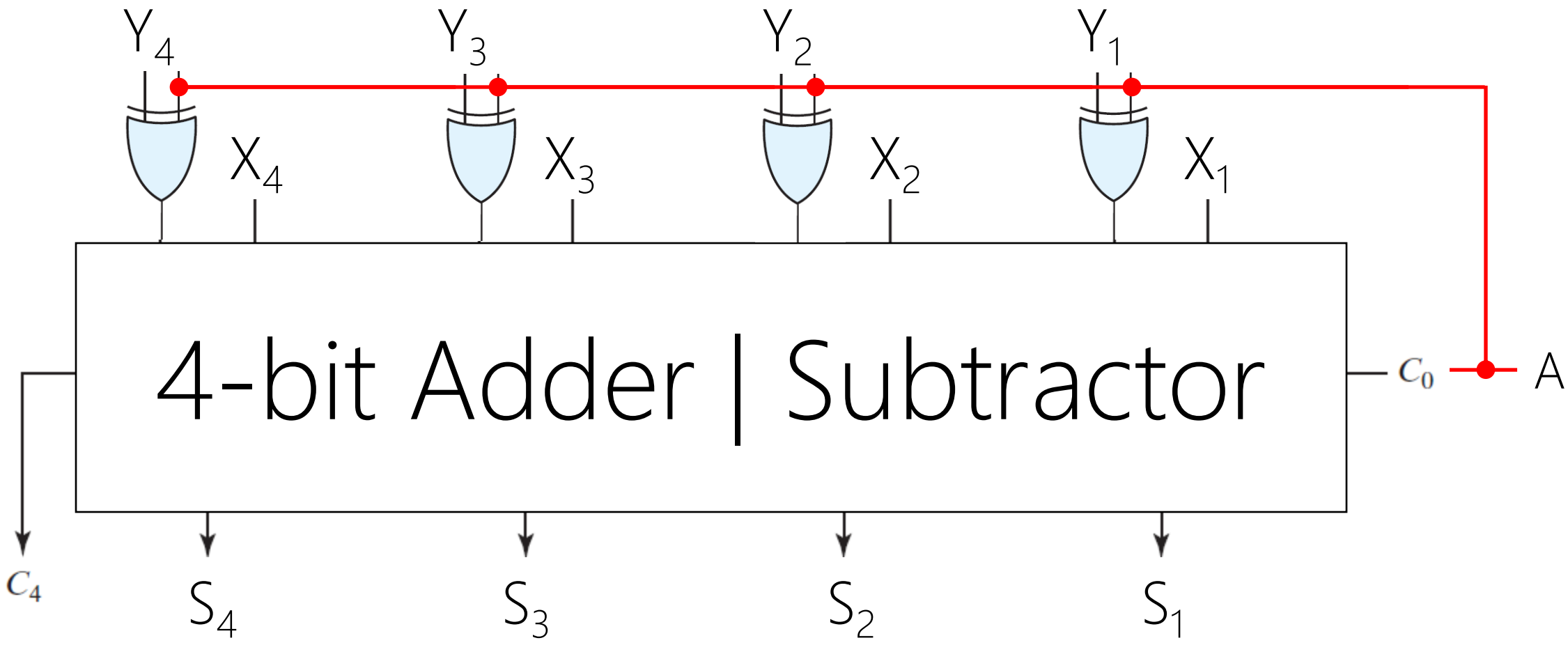

Y4
Y3
Y2
Y1
Y′4 X4
Y′3 X3
Y′2 X2
Y′1 X1
4-bit Adder | Subtractor
C0 = 0
= 1
C4
S4
S3
S2
S1


Y4
Y3
Y2
Y1
Y'4 X4
Y'3 X3
Y'2 X2
Y'1 X1
4-bit Adder | Subtractor
C0 = 0
= 1
C4
S4
S3
S2
S1


Y4
Y3
Y2
Y1
Y′4 X4
Y′3 X3
Y′2 X2
Y′1 X1
4-bit Adder | Subtractor
C0 = 0
= 1
C4
S4
S3
S2
S1

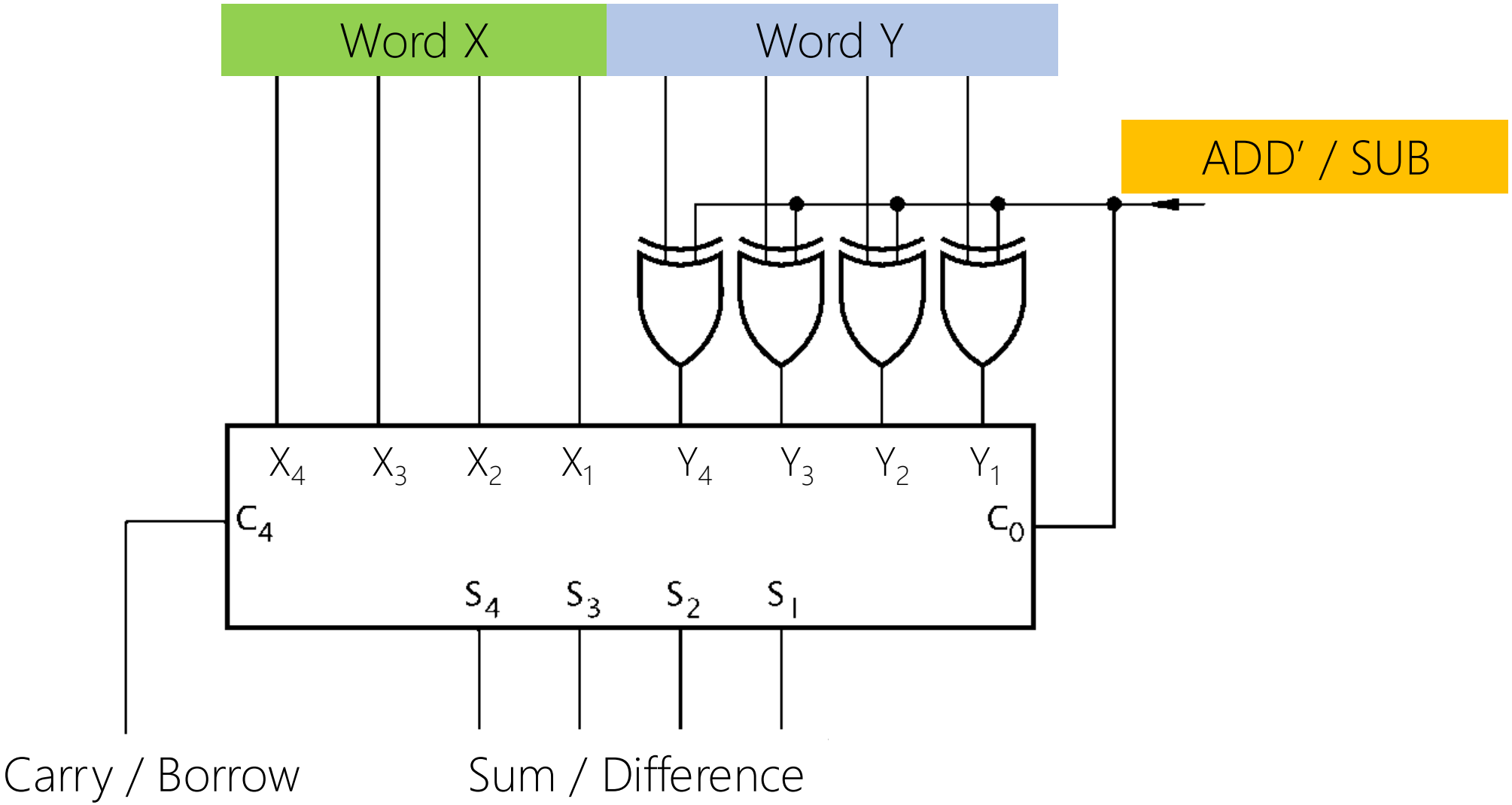
$$Y_i \; ? \; C_0 = \begin{cases} C_0 = \mathbf{0} \rightarrow Y_i \\ C_0 = \mathbf{1} \rightarrow Y'_i \end{cases}$$

$$Y_i \oplus C_0 = \begin{cases} C_0 = 0 \rightarrow Y_i \\ C_0 = 1 \rightarrow Y'_i \end{cases}$$



A=0 → Adder

A=1 → Subtractor


Word X
Word Y
ADD' / SUB
X4
X3
X2
X1
Y4
Y3
Y2
Y1
C4
C0
S4
S3
S2
S1
Carry / Borrow
Sum / Difference

Overflow

Signed-2's-Complement